KPR Institute of Engineering and Technology
(Autonomous)
Avinashi Road, Arasur, Coimbatore - 641 407

U21ADP05 – EXPLORATORY DATA ANALYSIS AND VISUALIZATION

III year B.Tech. Artificial Intelligence and Data Science

Assignment II – Case Study Report

From Clicks to Conversions: Exploratory Data Analysis and MLP-Based Prediction of Online Shoppers' Purchasing Intention

Nakul T (24AD068)

Date of Submission: 12/10/2026

Academic Year: 2026 - 27

ABSTRACT

Online retailers convert only a small share of visits into sales, so knowing which browsing behaviours precede a purchase has direct commercial value. This study analyses the Online Shoppers Purchasing Intention dataset (UCI, 12,330 sessions, 18 fields) through Ben Fry's seven stages of visualization. After removing 125 duplicate sessions and treating heavy-tailed variables with log transforms, univariate, distributional and multivariate analyses were carried out: page counts proved strongly over-dispersed (negative binomial rather than Poisson), browsing time followed a Weibull law with decreasing hazard, and PageValues, exit rate and month dominated the separation between buyers and non-buyers. A class-weighted multilayer perceptron, tuned over twelve configurations, reached a test ROC-AUC of 0.931 and PR-AUC of 0.740, outperforming logistic regression. The findings support PageValues-driven retargeting and season-aware campaign planning.

Keywords: exploratory data analysis; e-commerce conversion; probability distributions; Lorenz curve; multilayer perceptron; class imbalance; data visualization.

TABLE OF CONTENTS

1. INTRODUCTION AND OBJECTIVE 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Review of Related Literature 2
1.5 Organisation of the Report 3
2. DATASET DESCRIPTION 4
2.1 Source and Licence 4
2.2 Size and Structure 4
2.3 Basic Statistics 5
3. EXPLORATORY DATA ANALYSIS AND PREPROCESSING 6
3.1 Missing Values and Logical Consistency 6
3.2 Duplicate Sessions 6
3.3 Outlier Analysis 6
3.4 Transformation: Tukey's Ladder of Powers 7
3.5 Scaling and Standardisation 8
3.6 Feature Engineering and Encoding 8
3.7 Train, Validation and Test Split 8
3.8 Key Insights from EDA and Preprocessing 9
4. DATA VISUALIZATION 10
4.1 The Seven Stages of Visualization Applied 10
4.2 Univariate Distributions of All Numerical Features 10
4.3 Inequality of Engagement: Lorenz Curves and the Gini Coefficient 11
4.4 Discrete Distribution of Product Page Counts 12
4.5 Continuous Distribution of Browsing Time 13
4.6 Conversion as a Binomial Process across Months 14
4.7 Buyers versus Non-Buyers 15
4.8 Conversion Lift by Page Value and Exit Rate 16
4.9 Correlation Structure 17
4.10 A Resistant Line for Pages and Time 18
4.11 Categorical Associations and a Third Variable 18
4.12 Multivariate View: Pair Plot 20
4.13 Multivariate View: Principal Component Analysis 21
4.14 Interact: Exploratory Dashboard 22
4.15 Summary of Statistical Findings 23
5. DEEP LEARNING MODEL 24
5.1 Choice of Model 24
5.2 Architecture 24
5.3 Training Parameters 25
5.4 Hyperparameter Tuning 25
5.5 Feature Analysis and Importance 26

5.6 Baseline for Comparison....27
6. RESULT VISUALIZATION AND INTERPRETATION....28
6.1 Loss versus Epoch....28
6.2 Accuracy and ROC-AUC versus Epoch....29
6.3 Confusion Matrix....29
6.4 ROC Curve and AUC....30
6.5 Precision-Recall Curve and Threshold Analysis....31
6.6 Score Distribution and Error Analysis....31
6.7 Summary of Test Performance....32
7. CONCLUSION AND FUTURE SCOPE....33
7.1 What the Data Revealed....33
7.2 How the Model Performed....33
7.3 Challenges Faced....33
7.4 Recommendations for the Retailer....34
7.5 Future Scope....34
REFERENCES....35
APPENDIX A: PROJECT REPOSITORY AND REPRODUCIBILITY....36
APPENDIX B: SOURCE CODE (KEY SECTIONS)....37

LIST OF FIGURES

Figure 2.1: Composition of the raw dataset: (a) target balance, (b) visitor type and (c) sessions per month 5
Figure 3.1: Standardised box plots of six heavy-tailed features before and after the log1p transformation 7
Figure 3.2: Tukey's ladder of powers applied to product-related duration 8
Figure 4.1: Histograms of the ten numerical features (log1p scale for counts, durations and page values) 10
Figure 4.2: Lorenz curves and Gini coefficients of engagement and page value 11
Figure 4.3: Observed versus fitted Poisson and negative binomial frequencies, and the share of zero counts 12
Figure 4.4: Candidate continuous distributions for product-related duration and a normal Q-Q plot of its logarithm 13
Figure 4.5: Sessions per month and monthly conversion rate with 95% Wilson confidence intervals 14
Figure 4.6: Distributions of four key features for non-purchase and purchase sessions 15
Figure 4.7: Conversion rate across PageValues bands and ExitRates quintiles 16
Figure 4.8: Spearman rank-correlation matrix of the numerical features and the target 17
Figure 4.9: Log-log relationship between product pages and product time with Tukey's resistant line 18
Figure 4.10: Conversion rate by visitor type and day type (three-way contingency) 19
Figure 4.11: Pair plot of four key features coloured by class (3,000-session random sample) 20
Figure 4.12: Scree plot and PCA biplot of the ten log-transformed numerical features 21
Figure 4.13: Interactive EDA dashboard (dashboard/index.html) showing the PageValues view 22
Figure 5.1: Architecture of the selected multilayer perceptron 24
Figure 5.2: Validation ROC-AUC of the twelve hyperparameter configurations 26
Figure 5.3: Permutation importance of the trained MLP and model-free mutual information 26
Figure 6.1: Training and validation loss of the selected network 28
Figure 6.2: Accuracy and ROC-AUC per epoch for training and validation data 29
Figure 6.3: Test-set confusion matrices at the default and the validation-tuned threshold 29
Figure 6.4: ROC curves of the MLP and the logistic-regression baseline on the test set 30
Figure 6.5: Precision-recall curves and precision, recall and F1 as functions of the threshold 31
Figure 6.6: Distribution of predicted purchase probability by true class (test set, log scale) 31

LIST OF TABLES

Table 2.1: Data dictionary of the eighteen fields 4
Table 2.2: Five-number summary, mean, standard deviation, skewness and share of zeros (12,205 sessions) 5
Table 3.1: Consistency between page counts and durations (raw data) 6
Table 3.2: Outlier screening with Tukey's fences and the z-score rule 7
Table 3.3: Feature engineering and encoding steps 8
Table 3.4: Stratified train, validation and test split 9
Table 4.1: Fry's seven stages mapped to the work in this project 10
Table 4.2: Maximum-likelihood fits to the ProductRelated page count 12
Table 4.3: Maximum-likelihood fits to positive product-related duration (11,575 sessions) 13
Table 4.4: Mann-Whitney U tests between buyers and non-buyers (all $p < 10^{-21}$ after Benjamini-Hochberg correction) 15
Table 4.5: Chi-square tests of independence between categorical variables and purchase 18
Table 4.6: Summary of the statistical and distribution-based analysis 23
Table 5.1: Layer-by-layer summary of the selected network 24
Table 5.2: Training configuration 25
Table 5.3: Hyperparameter search results (selected configuration: #1) 25
Table 6.1: Test-set performance of the MLP and the logistic-regression baseline 32

1. INTRODUCTION AND OBJECTIVE

1.1 Background

Every visit to an online store leaves a trail of page requests, dwell times and navigation signals that web-analytics platforms such as Google Analytics summarise into a handful of session-level metrics. Only a minority of these visits end in a purchase, so a retailer that can recognise purchase intent while the visitor is still on the site can personalise offers, prioritise customer support or save marketing budget on visitors who were never going to buy. Turning raw session metrics into that kind of knowledge is the business of exploratory data analysis (EDA): looking at the data from many angles, with numerical summaries and graphics, before committing to a model [4].

This case study works with the Online Shoppers Purchasing Intention dataset donated to the UCI Machine Learning Repository by Sakar and Kastro [2] and introduced in a study of real-time purchase prediction [1]. It contains 12,330 browsing sessions of an online retail site collected over one year, with each session belonging to a different user. Only 1,908 of those sessions (15.5%) ended in a transaction, an imbalance that influences almost every decision in the chapters that follow, from the choice of evaluation metric to the way the neural network is trained.

1.2 Problem Statement

Given the aggregated behaviour of a visitor during one session (pages viewed by type, time spent, the bounce, exit and value metrics of those pages, and contextual information such as month, visitor type and traffic source), the study asks two questions. First, how do purchasing and non-purchasing sessions differ, and which statistical distributions describe the behavioural variables? Second, can a deep learning model predict whether a session will end in a purchase accurately enough to support a real-time intervention?

1.3 Objectives

1. Audit the data for explicit and implicit missing values, logical inconsistencies, duplicates and outliers, and prepare it for modelling without leaking information from the test set into training.
2. Describe every numerical variable through measures of level, spread, shape and inequality (five-number summaries, skewness, Lorenz curves and the Gini coefficient).
3. Identify discrete and continuous probability distributions for page counts and browsing time, and treat conversion as a binomial proportion whose rate changes with the month.
4. Analyse bivariate and multivariate structure with rank tests, contingency tables, rank correlation, a resistant line, a third-variable control and principal component analysis.

5. Present purposeful visualizations that follow Fry's seven stages, including an interactive dashboard.
6. Build, tune and evaluate a multilayer perceptron, and interpret it with learning curves, a confusion matrix, ROC and precision-recall analysis and permutation feature importance.
7. Translate the findings into recommendations for an online retailer and identify future improvements.

1.4 Review of Related Literature

Research on online conversion predates modern deep learning. Moe and Fader [5] modelled conversion as a dynamic process in which a shopper's history of visits changes the probability that the current visit ends in a purchase, establishing that behaviour observed during visits carries real information about intent. Bucklin and Sismeiro [6] estimated a model of browsing directly from clickstream logs, describing both the decision to request another page and the time spent on each page, two quantities that appear here as page counts and durations. These studies motivate the statistical treatment of counts and durations in Chapter 4.

Sakar et al. [1] built a real-time system on the present dataset. Aggregated page-view features and session information were fed to random forest, support vector machine and multilayer perceptron classifiers, with oversampling and feature selection used to handle class imbalance. The multilayer perceptron gave significantly higher accuracy and F1 score than the other two classifiers, and combining clickstream features with session-information features improved the results. A second module used a long short-term memory network on sequential clickstream data to estimate the likelihood of leaving the site without buying.

On the methodological side, Fry [3] describes visualization as a seven-stage process (acquire, parse, filter, mine, represent, refine and interact), which structures Chapter 4. Goodfellow et al. [7] provide the theory of feed-forward networks, while Adam optimisation [8] and dropout regularisation [9] are the training choices adopted in Chapter 5. Because only one session in six converts, Fawcett's treatment of ROC analysis [11] is complemented by Saito and Rehmsmeier's argument that precision-recall curves are more informative on imbalanced data [10]. Count-data theory [12], the Wilson score interval [13] and permutation importance [14] supply the remaining statistical tools.

Most published analyses of this dataset concentrate on classifier accuracy. Fewer document the statistical shape of the variables (over-dispersion, zero inflation and highly unequal engagement), even though that shape determines how the data should be cleaned, transformed and evaluated. This report puts that exploratory evidence first and lets it drive the model design.

1.5 Organisation of the Report

Chapter 2 describes the dataset and its basic statistics. Chapter 3 documents the data-quality audit and every preprocessing step. Chapter 4 presents the visual and statistical analysis, organised by Fry's seven stages. Chapter 5 explains the multilayer perceptron, its training parameters and its tuning, and Chapter 6 interprets the results on unseen test data. Chapter 7 concludes with recommendations and future scope. The appendices give the repository structure and the key source code.

2. DATASET DESCRIPTION

2.1 Source and Licence

The data were obtained from the UCI Machine Learning Repository as dataset 468, the Online Shoppers Purchasing Intention Dataset [2] (DOI 10.24432/C5F88Q), released under the Creative Commons Attribution 4.0 licence. It is a single comma-separated file of 1.07 MB. The sessions were sampled so that each belongs to a different user over a one-year period, which avoids any bias towards a particular campaign, special day, user profile or period [1].

2.2 Size and Structure

The raw file holds **12,330 rows and 18 columns**: ten numerical behavioural variables, seven categorical or Boolean context variables, and the binary target `Revenue`. The page-count and duration variables come in three families (administrative, informational and product-related pages). `BounceRates`, `ExitRates` and `PageValues` are Google Analytics metrics averaged over the pages a visitor saw. Only ten months are present: January and April do not appear, and June is spelled out in full while the other months are abbreviated. Both details matter for encoding and for interpreting the monthly analysis.

Table 2.1: Data dictionary of the eighteen fields

Field	Type	Description
Administrative	Count	Account-management pages visited
Administrative_Duration	Continuous (s)	Time spent on administrative pages
Informational	Count	Pages about the site, contact or address
Informational_Duration	Continuous (s)	Time spent on informational pages
ProductRelated	Count	Product-related pages visited
ProductRelated_Duration	Continuous (s)	Time spent on product-related pages
BounceRates	Rate (0 to 0.2)	Mean bounce rate of the pages visited
ExitRates	Rate (0 to 0.2)	Mean exit rate of the pages visited
PageValues	Continuous	Mean value of pages visited before a transaction
SpecialDay	Ordinal (0 to 1)	Closeness of the visit to a special day such as Mother's Day
Month	Categorical (10)	Month of the visit (January and April absent)
OperatingSystems	Categorical (8)	Operating system code
Browser	Categorical (13)	Browser code
Region	Categorical (9)	Geographic region code
TrafficType	Categorical (20)	Traffic-source code (search, referral, direct and so on)
VisitorType	Categorical (3)	Returning, New or Other visitor
Weekend	Boolean	Whether the session fell on a weekend
Revenue	Boolean (target)	Whether the session ended in a purchase

2.3 Basic Statistics

Table 2.2 summarises the numerical variables after removing duplicates (Section 3.2). For every behavioural variable the mean lies far above the median, a first sign of strong right skew. Several variables are dominated by zeros: 78% of sessions have no page value, 78% view no informational page and 90% are not close to a special day.

Table 2.2: Five-number summary, mean, standard deviation, skewness and share of zeros (12,205 sessions)

Feature	Min	Q1	Median	Q3	Max	Mean	SD	Skew	Zeros %
Administrative	0.000	0.000	1.00	4.00	27.00	2.34	3.33	1.95	46.2
Administrative_Dur.	0.000	0.000	9.00	94.70	3,399	81.65	177	5.59	47.3
Informational	0.000	0.000	0.000	0.000	24.00	0.509	1.28	4.01	78.4
Informational_Dur.	0.000	0.000	0.000	0.000	2,549	34.83	141	7.54	80.3
ProductRelated	0.000	8.00	18.00	38.00	705	32.05	44.59	4.33	0.3
ProductRelated_Dur.	0.000	193	609	1,477	63,974	1,207	1,920	7.25	5.2
BounceRates	0.000	0.000	0.003	0.017	0.200	0.020	0.045	3.16	45.2
ExitRates	0.000	0.014	0.025	0.049	0.200	0.041	0.046	2.23	0.6
PageValues	0.000	0.000	0.000	0.000	362	5.95	18.65	6.35	77.6
SpecialDay	0.000	0.000	0.000	0.000	1.00	0.062	0.200	3.29	89.8

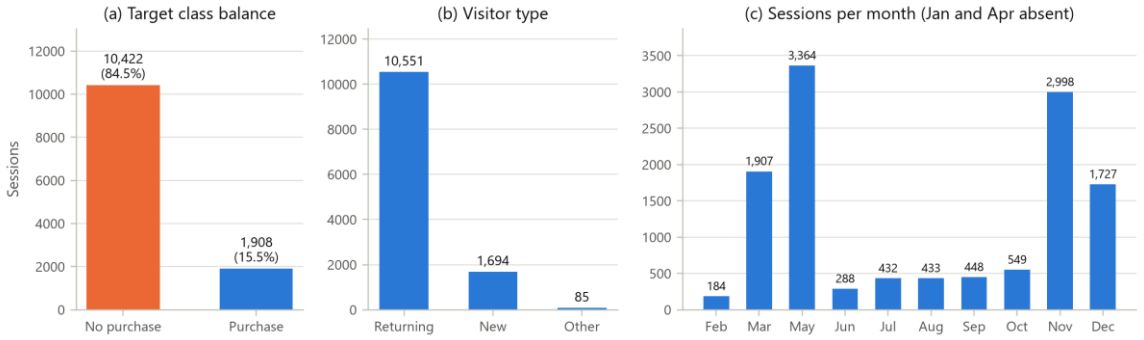


Figure 2.1: Composition of the raw dataset: (a) target balance, (b) visitor type and (c) sessions per month

Figure 2.1 shows the three facts that shape the analysis. Purchases form a minority class of 15.5%. Returning visitors dominate the traffic (10,551 sessions, 85.6%). Activity is concentrated in May, November, March and December, which together account for 81% of all sessions.

3. EXPLORATORY DATA ANALYSIS AND PREPROCESSING

This chapter covers the filter stage of the visualization process: checking that the data can be trusted, then reshaping it into a form suitable for statistics and modelling. Every rule below was derived from evidence in the data, and every statistic used in modelling (scaling constants, rare-category lists, class weights) was learned from the training split only.

3.1 Missing Values and Logical Consistency

A direct count found **0 missing cells**: the dataset has no explicit gaps. Missing information can still hide inside valid-looking values, so three logical checks were added. First, a page count of zero should never come with a positive duration. Second, rates must lie between 0 and 0.2, which is the range Google Analytics produces for these averages. Third, no variable should be negative. All three checks passed.

Table 3.1: Consistency between page counts and durations (raw data)

Page family	Count = 0, duration > 0	Count > 0, duration = 0
Administrative	0	135
Informational	0	226
ProductRelated	0	717

Table 3.1 shows 1,078 cases (135 administrative, 226 informational and 717 product-related) in which pages were visited but no time was recorded. This is not data loss. Analytics tools measure time on a page as the gap between that page's hit and the next hit in the same session, so the final page of a session (and the only page of a single-page visit) is always recorded as zero seconds. These structural zeros were kept because they carry real information: they mark short, shallow sessions.

3.2 Duplicate Sessions

125 rows (1.01%) were exact duplicates across all eighteen fields. Profiling them showed a single pattern: 100% had a bounce rate of 0.2, 100% recorded zero product-related time, 94.4% viewed at most one product page and 0% ended in a purchase. They are identical single-page bounce visits. Keeping them would place copies of the same row in both the training and the test set, which inflates test scores, and would over-weight bounce behaviour. They were removed, leaving **12,205 sessions** with a purchase rate of 15.63%.

3.3 Outlier Analysis

Two common rules were compared. Tukey's fences flag values beyond 1.5 interquartile ranges from the quartiles, and the z-score rule flags values more than three standard deviations from the mean. Both were applied before and after a log1p transformation, $\log(1 + x)$, which accepts zeros.

Table 3.2: Outlier screening with Tukey's fences and the z-score rule

Feature	Q1	Q3	Upper fence	IQR outliers	%	|z|>3 raw	|z|>3 log1p
Administrative	0.000	4.00	10.00	404	3.3	213	0
Administrative_Dur.	0.000	94.70	237	1,149	9.4	230	0
Informational	0.000	0.000	0.000	2,631	21.6	260	260
Informational_Dur.	0.000	0.000	0.000	2,405	19.7	229	177
ProductRelated	8.00	38.00	83.00	1,007	8.3	236	6
ProductRelated_Dur.	193	1,477	3,403	951	7.8	217	630
BounceRates	0.000	0.017	0.042	1,428	11.7	593	596
ExitRates	0.014	0.049	0.100	1,325	10.9	599	595
PageValues	0.000	0.000	0.000	2,730	22.4	257	125
SpecialDay	0.000	0.000	0.000	1,249	10.2	478	478

Table 3.2 shows that the interquartile rule breaks down on zero-inflated variables. For `Informational`, `PageValues` and `SpecialDay` the first and third quartiles are both zero, so every non-zero value counts as an outlier (up to 22% of sessions). The z-rule is more conservative, and after the log transformation most of its flags disappear (from 236 to 6 for `ProductRelated`, and to zero for the administrative variables). The 630 flags that remain for product-related duration are the zero-time sessions, which the logarithm moves into the lower tail. They are structural zeros rather than errors.

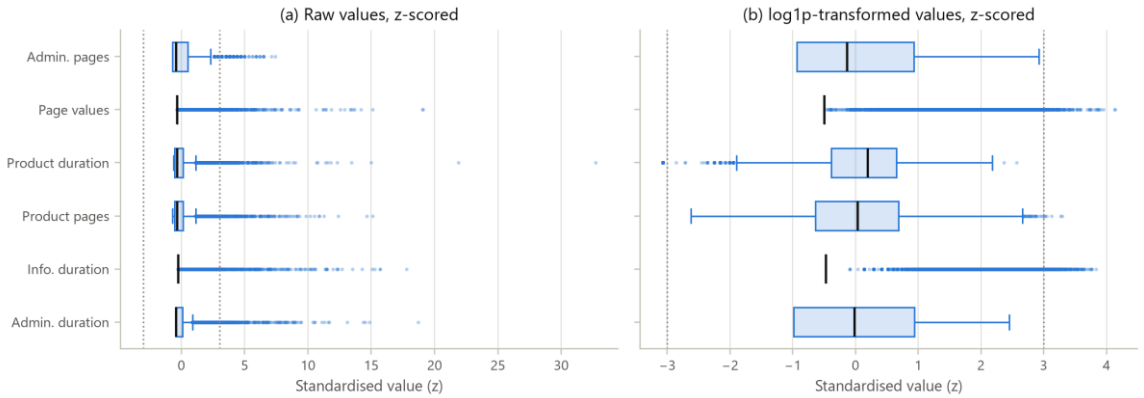


Figure 3.1: Standardised box plots of six heavy-tailed features before and after the log1p transformation

Figure 3.1 makes the decision visible. On the raw scale (a) the whiskers are squeezed against zero and the outliers stretch beyond z = 30, whereas after the transformation (b) the boxes are readable and the tails are bounded. No session was deleted. The extreme sessions are real shoppers, and they are disproportionately buyers: the median product-browsing time of buyers is 1,110 s against 526 s for non-buyers. Removing them would bias the target. The log1p transformation followed by standardisation was used instead.

3.4 Transformation: Tukey's Ladder of Powers

Tukey's ladder of powers re-expresses a variable as x^p for p = 1, 1/2, 0 (the logarithm) and -1/2, and the rung that makes the distribution most symmetric is chosen [4]. Figure 3.2 applies the ladder to the 11,575 positive product-related durations.

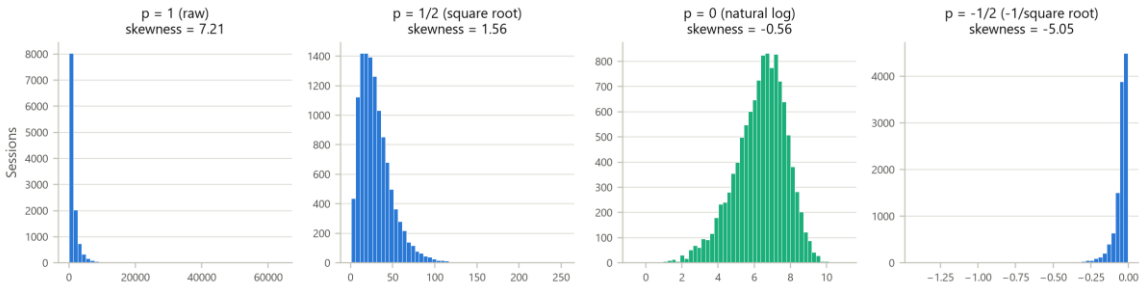


Figure 3.2: Tukey's ladder of powers applied to product-related duration

Skewness falls from 7.21 on the raw scale to 1.56 for the square root and -0.56 for the logarithm, then overshoots to -5.05 at p = -1/2. The logarithm is the rung closest to symmetry, so a log transformation (log1p, to accommodate zeros) was applied to all ten count and duration variables, including the engineered ones described below.

3.5 Scaling and Standardisation

Neural networks train best when inputs share a common scale. The fifteen numerical model inputs were therefore standardised with z-scores. The mean and standard deviation were estimated on the training split only and then applied unchanged to the validation and test splits:

$$z = (x - \bar{x}_{\text{train}}) \,/\, s_{\text{train}}$$

3.6 Feature Engineering and Encoding

Table 3.3: Feature engineering and encoding steps

Step	Details
New features	TotalPages, TotalDuration, DurationPerProductPage (product time per product page) and HasPageValue (1 if PageValues > 0)
Text cleaning	Month label 'June' renamed to 'Jun' for consistency
Rare categories	Levels used by fewer than 1% of training sessions pooled into 'Other' (OperatingSystems, Browser, Region, TrafficType)
Encoding	One-hot encoding of Month, OperatingSystems, Browser, Region, TrafficType and VisitorType; Weekend as 0/1
Transformation	log1p on ten count and duration variables, then z-score scaling
Final input	62 columns: 15 scaled numerical + 47 one-hot indicators

3.7 Train, Validation and Test Split

The 12,205 sessions were split 70/15/15 with stratification on the target, so that each subset keeps the same purchase rate (Table 3.4). The validation set is used for early stopping, hyperparameter selection and the choice of decision threshold. The test set was touched only once, for the final evaluation in Chapter 6.

Table 3.4: Stratified train, validation and test split

Subset	Sessions	Purchases	Purchase rate (%)
Train	8,543	1,336	15.64
Validation	1,831	286	15.62
Test	1,831	286	15.62

To counter the 84:16 imbalance, each class was weighted in the loss function by $n / (2\ n_c)$, giving weights of 0.593 for non-purchase and 3.197 for purchase sessions. A missed buyer therefore costs about 5.4 times as much as a false alarm during training.

3.8 Key Insights from EDA and Preprocessing

- The data are complete but not clean: 125 duplicate bounce sessions and 1,078 structural zero durations had to be understood before any modelling.
- Every behavioural variable is right-skewed (skewness 1.9 to 7.5) and several are zero-inflated, so means and standard deviations alone are misleading. Medians, IQRs and log scales are needed.
- Interquartile-range outlier rules are unsuitable for zero-inflated data. Transformation, not deletion, is the right treatment for this dataset.
- The log transformation is the empirically best rung of Tukey's ladder and reduces skewness from 7.21 to -0.56.

4. DATA VISUALIZATION

Each visualization in this chapter answers a specific analytical question. They are grouped by the analysis they support: univariate shape and inequality, probability distributions, bivariate comparison, multivariate structure and, finally, interaction. Every chart is followed by what it shows, why it was created and the insight it produced. All charts use a colour-blind-safe palette: blue always marks purchase sessions and orange marks non-purchase sessions.

4.1 The Seven Stages of Visualization Applied

Table 4.1: Fry's seven stages mapped to the work in this project

Stage [3]	How it was carried out in this study
1. Acquire	Downloaded dataset 468 from the UCI repository; stored the CSV in the project's data folder
2. Parse	Assigned data types and roles (count, continuous, rate, categorical, target); built the data dictionary
3. Filter	Removed 125 duplicates; verified consistency rules; diagnosed outliers; pooled rare categories
4. Mine	Summary statistics, distribution fitting, Mann-Whitney and chi-square tests, Spearman correlation, resistant line, PCA, multilayer perceptron
5. Represent	Chose a chart form per question: histograms, Lorenz curves, box plots, heat maps, hexbin, pair plot, biplot, ROC and precision-recall curves
6. Refine	Log scales for skewed data, colour-blind-safe palette, direct labels, confidence intervals, consistent class colours
7. Interact	Self-contained HTML dashboard with feature, scale, visitor-type, day-type and month filters (Section 4.14)

4.2 Univariate Distributions of All Numerical Features

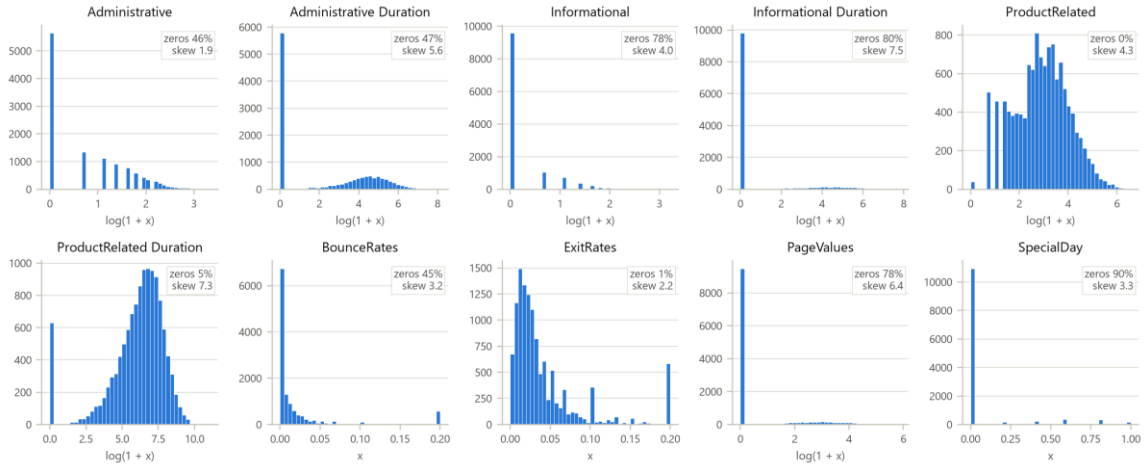


Figure 4.1: Histograms of the ten numerical features (log1p scale for counts, durations and page values)

What it shows. Ten small histograms, one per numerical feature, each annotated with its share of zeros and its skewness. Counts, durations and page values are drawn on the log1p scale; the three bounded rates are on their natural scale.

Why it was created. Univariate exploration is the starting point for any EDA. Small multiples allow the shapes to be compared at a glance and show where a common transformation will work.

Insights obtained. Two kinds of shape appear. Variables with a mass at zero and a separate hump (administrative and informational pages, page values) are zero-inflated mixtures: most sessions never touch these pages, and the minority that do behave very differently. ProductRelated and its duration are close to bell-shaped on the log scale, so they are roughly lognormal. Exit and bounce rates show spikes at 0.2, the value produced by single-page sessions.

4.3 Inequality of Engagement: Lorenz Curves and the Gini Coefficient

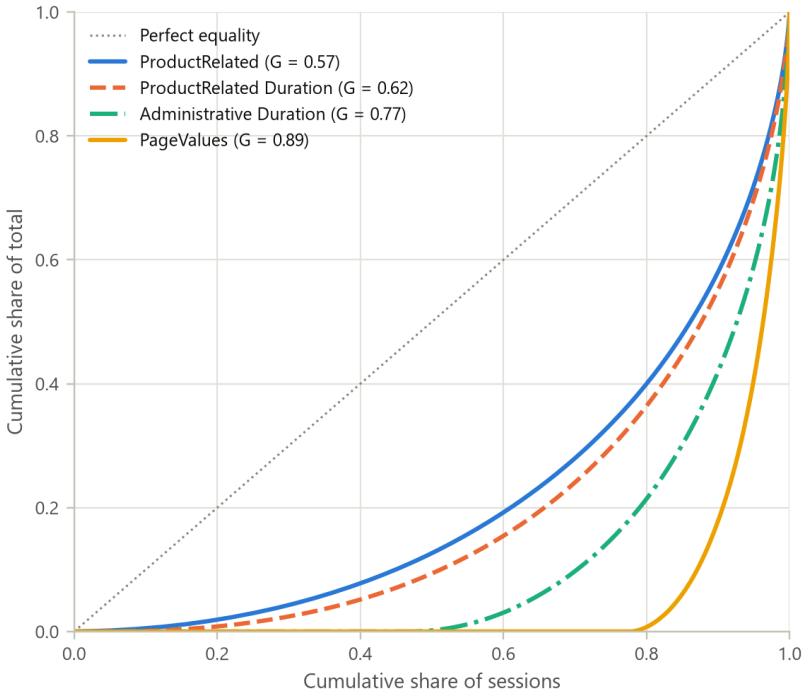


Figure 4.2: Lorenz curves and Gini coefficients of engagement and page value

What it shows. Each curve plots the cumulative share of sessions (sorted from least to most engaged) against the cumulative share of the total quantity they account for. The diagonal represents perfect equality.

Why it was created. Inequality measures from the univariate-analysis toolkit show how concentrated a quantity is, which a mean or standard deviation cannot show.

Insights obtained. Engagement is highly concentrated. The top 10% of sessions account for 44.8% of all product-browsing time (Gini 0.62) and 42.0% of product page views (Gini 0.57). Page value is the most unequal quantity (Gini 0.89): 82.1% of it sits in the top tenth of sessions, which follows from the fact that only 22.4% of sessions reach any valued page at all. Commercially, a small, identifiable group of sessions carries most of the value.

4.4 Discrete Distribution of Product Page Counts

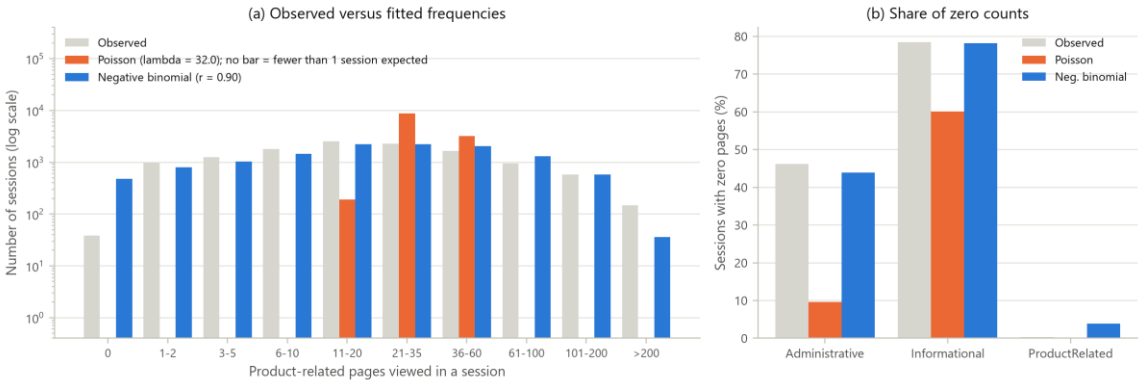


Figure 4.3: Observed versus fitted Poisson and negative binomial frequencies, and the share of zero counts

Table 4.2: Maximum-likelihood fits to the ProductRelated page count

Model	Parameters	Log-likelihood	AIC	Chi-square (10 bins)
Poisson	lambda = 32.05	-261,158	522,318	3.7×10^{14}
Negative binomial	r = 0.899, p = 0.0273	-54,669	109,341	1,132

What it shows. Panel (a) compares the observed number of sessions in ten page-count bins with the frequencies expected under a Poisson model and under a negative binomial model, on a log scale. Panel (b) compares the observed share of zero counts for the three page families with each model's prediction.

Why it was created. Page counts are discrete, so a count distribution is the natural first hypothesis. The Poisson model assumes that the variance equals the mean, and this chart tests that assumption visually and numerically.

Insights obtained. The Poisson assumption fails badly: the variance of ProductRelated is 1,988 against a mean of 32.0, a dispersion index of 62. A Poisson model with the same mean puts almost all sessions between 21 and 60 pages and predicts essentially no sessions below 11 or above 60 pages, whereas thousands are observed. The negative binomial, which adds a heterogeneity parameter r, lowers the AIC from 522,318 to 109,341 and reproduces both tails. Panel (b) shows the same effect for zeros: 46.2% of sessions visit no administrative page, while a Poisson model predicts 9.6% and the negative binomial predicts 44.0%. Shoppers differ so much from one another that a single shared rate cannot describe them; the chi-square statistic remains significant because of the large sample, so the negative binomial should be read as a much better description rather than a perfect one.

4.5 Continuous Distribution of Browsing Time

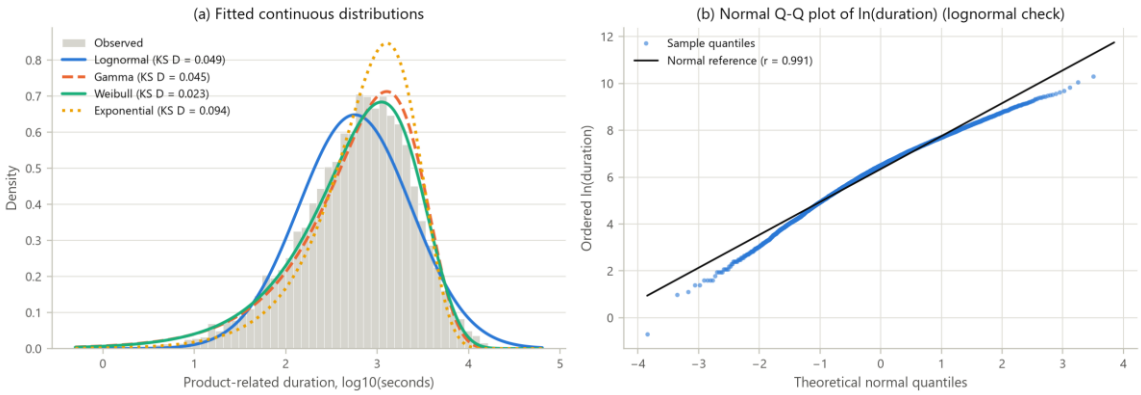


Figure 4.4: Candidate continuous distributions for product-related duration and a normal Q-Q plot of its logarithm

Table 4.3: Maximum-likelihood fits to positive product-related duration (11,575 sessions)

Distribution	Shape	Scale (s)	AIC	KS distance D
Weibull	k = 0.807	1,116	187,569	0.0233
Lognormal	sigma = 1.417	570	187,817	0.0487
Gamma	k = 0.746	1,707	187,914	0.0455
Exponential	-	1,273	188,648	0.0942

What it shows. Panel (a) overlays four fitted densities (Weibull, lognormal, gamma and exponential) on the histogram of browsing time drawn on a log axis. Panel (b) is a normal quantile-quantile plot of log-duration, which is a direct test of the lognormal hypothesis.

Why it was created. Durations are positive and right-skewed, so the classical candidate families were fitted by maximum likelihood and compared with the Akaike information criterion (AIC) and the Kolmogorov-Smirnov distance.

Insights obtained. The Weibull distribution fits best on both criteria (AIC 187,569, D = 0.023). Its shape parameter k = 0.81 is below 1, which means a decreasing hazard: the longer a shopper has already browsed, the less likely the session is to end in the next moment, so engaged sessions tend to stay engaged. The lognormal comes second. Its Q-Q plot is nearly straight (r = 0.991) but bends below the line at the lower end because log-duration is left-skewed (skewness -0.56): there are more very short sessions than a lognormal predicts. The exponential, which has a constant hazard, fits worst. The typical session spends about 570 s (the geometric mean) on product pages.

4.6 Conversion as a Binomial Process across Months

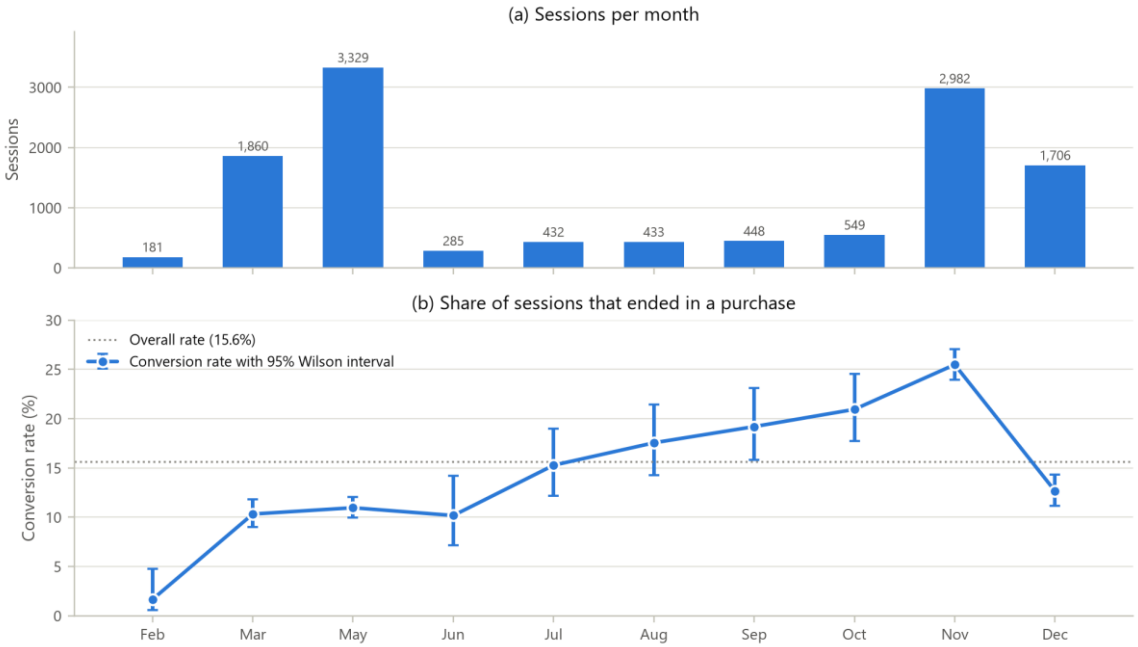


Figure 4.5: Sessions per month and monthly conversion rate with 95% Wilson confidence intervals

What it shows. Panel (a) gives the number of sessions in each month; panel (b) gives the share of those sessions that ended in a purchase, with a 95% Wilson score interval [13] for each month and the overall rate as a reference line. The two measures are drawn on separate aligned panels instead of a dual-axis chart, so neither scale distorts the other.

Why it was created. Each session is a Bernoulli trial with outcome purchase or no purchase, so the number of purchases in a month is binomial. The Wilson interval is accurate even for small months such as February, where the normal approximation fails.

Insights obtained. The overall conversion probability is 15.63% (95% CI 15.00% to 16.29%), but the monthly rates differ far more than chance allows (chi-square = 376.3, df = 9, $p < 10^{-74}$). November converts best (25.5%, CI 23.9% to 27.1%), followed by October (20.9%) and September (19.2%). February converts worst (1.7%). May has the most traffic but converts at only 11.0%. Volume and conversion are separate, so a busy month is not necessarily a profitable one. The rise from summer towards November is consistent with pre-holiday shopping.

4.7 Buyers versus Non-Buyers

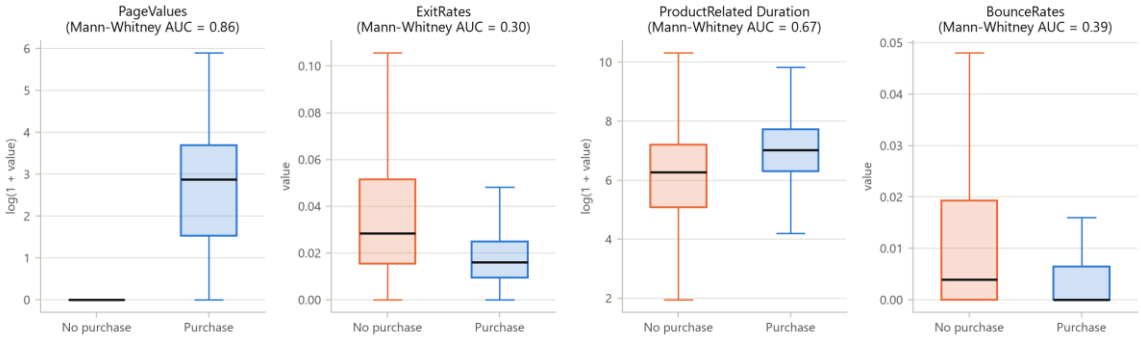


Figure 4.6: Distributions of four key features for non-purchase and purchase sessions

Table 4.4: Mann-Whitney U tests between buyers and non-buyers (all $p < 10^{-21}$ after Benjamini-Hochberg correction)

Feature	Median (buyers)	Median (non-buyers)	AUC effect	Rank-biserial r
PageValues	16.8	0.000	0.863	+0.726
ExitRates	0.016	0.028	0.300	-0.399
ProductRelated_Dur.	1,109.9	526.0	0.669	+0.338
ProductRelated	29.0	16.0	0.654	+0.309
Administrative	2.00	1.00	0.624	+0.248
Administrative_Dur.	52.4	0.000	0.621	+0.241
BounceRates	0.000	0.004	0.391	-0.218
Informational	0.000	0.000	0.564	+0.128
Informational_Dur.	0.000	0.000	0.561	+0.122
SpecialDay	0.000	0.000	0.463	-0.074

What it shows. Side-by-side box plots compare the two classes on four features, each panel titled with its Mann-Whitney effect size. Table 4.4 extends the comparison to all ten numerical features.

Why it was created. Comparing several batches with box plots shows location, spread and skew together. The rank-based Mann-Whitney test is appropriate because none of the variables is normal. Its AUC form, the probability that a random buyer scores higher than a random non-buyer, is easy to interpret.

Insights obtained. PageValues is by far the strongest single signal (AUC 0.863). Most non-buyers have a page value of zero, while the median buyer has 16.8. Buyers also leave fewer pages as exits (median exit rate 0.016 against 0.028, AUC 0.30, which means lower for buyers) and spend roughly twice as long on product pages (1,110 s against 526 s). Bounce rate and SpecialDay are also lower for buyers (AUC 0.39 and 0.46); the latter suggests that sessions close to a special day attract browsing rather than buying.

4.8 Conversion Lift by Page Value and Exit Rate

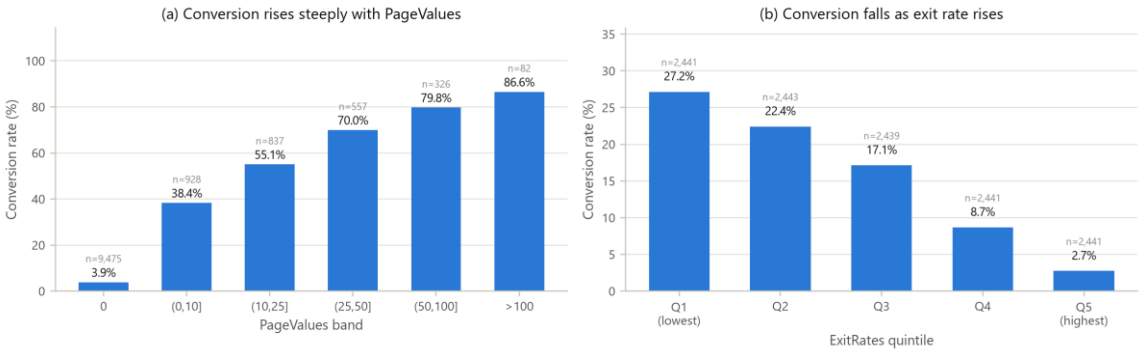


Figure 4.7: Conversion rate across PageValues bands and ExitRates quintiles

What it shows. Bar charts of the conversion rate within bands of PageValues and within quintiles of ExitRates, with the number of sessions in each group printed above the bar.

Why it was created. Turning a continuous predictor into ordered bands shows the shape of its relationship with the outcome (monotone, threshold or plateau) in a form that business users can act on.

Insights obtained. Conversion rises steadily with page value: 3.9% for sessions with no page value, 38.4% for values up to 10, 70.0% for 25 to 50 and 86.6% above 100. Simply reaching a valued page raises the conversion rate almost tenfold. Exit rate works in the opposite direction, from 27.2% in the lowest quintile to 2.7% in the highest. Together the two charts give a simple rule a retailer can use.

4.9 Correlation Structure

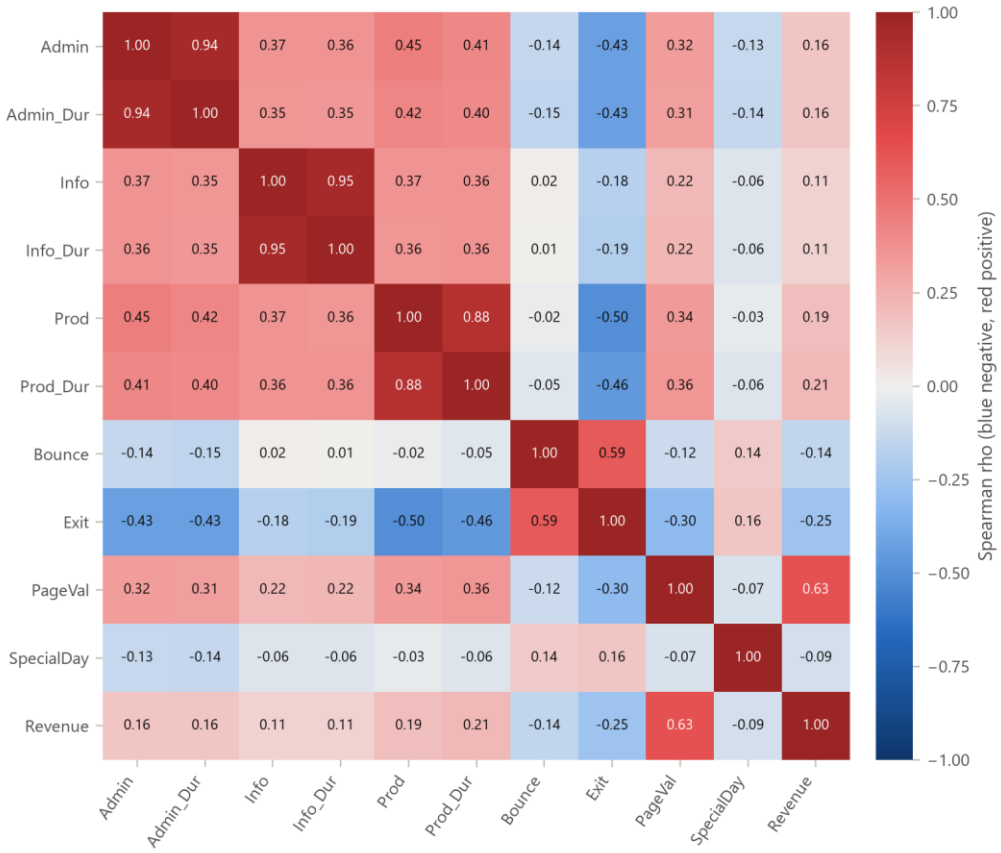


Figure 4.8: Spearman rank-correlation matrix of the numerical features and the target

What it shows. A matrix of Spearman rank correlations on a diverging blue-to-red scale (blue negative, red positive, grey near zero) with the coefficient printed in every cell.

Why it was created. Rank correlation is robust to the skew and the zero-inflation documented above. The matrix reveals redundant inputs (multicollinearity) and the variables most associated with the target.

Insights obtained. Each count is almost interchangeable with its own duration (rho = 0.94, 0.95 and 0.88), and bounce and exit rates move together (rho = 0.59). This redundancy motivates both the use of PCA in Section 4.13 and regularisation in the network. Revenue is most correlated with PageValues (rho = 0.63), negatively correlated with ExitRates (-0.25) and only weakly correlated with page counts (0.19). How much a visitor browses matters less than where the browsing leads.

4.10 A Resistant Line for Pages and Time

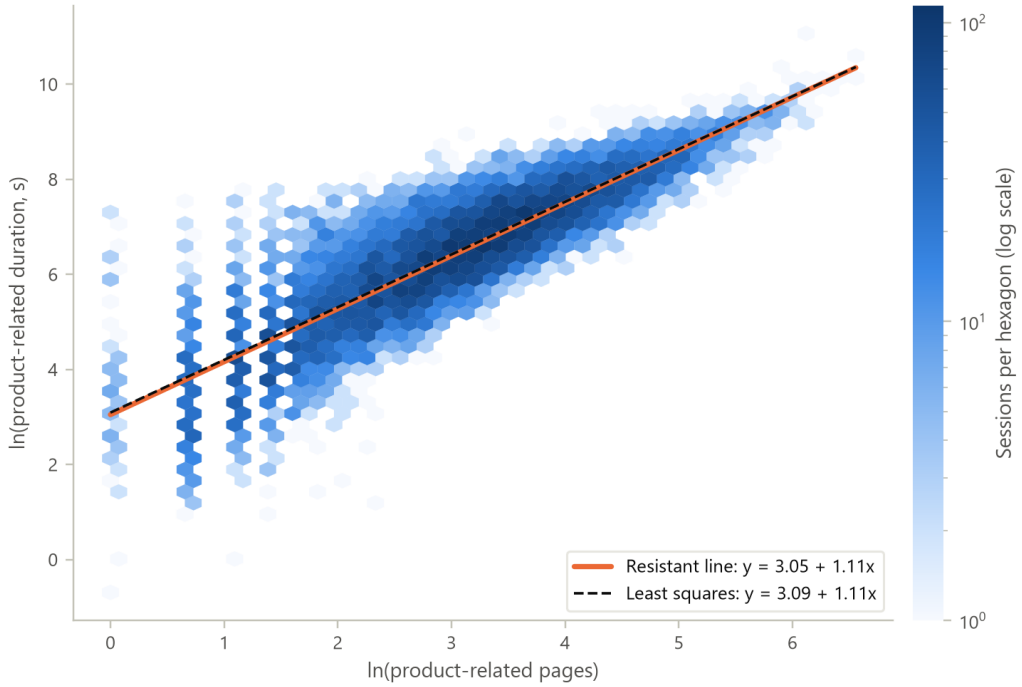


Figure 4.9: Log-log relationship between product pages and product time with Tukey's resistant line

What it shows. A hexagonal-bin density plot of log product time against log product pages for 11,575 sessions, with Tukey's three-group resistant line and the least-squares line overlaid.

Why it was created. Scatterplots with resistant lines are a classic bivariate EDA tool. Hexagonal binning avoids the overplotting of 11,000 points, and the resistant line, built from the medians of three groups, cannot be pulled around by extreme sessions.

Insights obtained. The relationship is strongly linear on the log-log scale (Spearman rho = 0.86). The resistant line y = 3.05 + 1.11x and the least-squares line y = 3.09 + 1.11x almost coincide, so the extreme sessions do not distort the fit. A slope slightly above 1 means that time grows a little faster than pages: doubling the number of product pages multiplies browsing time by $2^{1.11}$, about 2.16. At the median session the line implies roughly 30 s per product page.

4.11 Categorical Associations and a Third Variable

Table 4.5: Chi-square tests of independence between categorical variables and purchase

Variable	Levels	Chi-square	df	p-value	Cramer's V
Month	10	376.3	9	$< 10^{-74}$	0.176
TrafficType	12	342.3	11	$< 10^{-65}$	0.167
VisitorType	3	130.7	2	$< 10^{-28}$	0.103
SpecialDay	6	98.2	5	$< 10^{-18}$	0.090
OperatingSystems	5	73.2	4	$< 10^{-14}$	0.077
Browser	8	11.1	7	0.135	0.030
Region	9	9.7	8	0.288	0.028
Weekend	2	9.2	1	0.002	0.027

Table 4.5 ranks the categorical context variables. Month (V = 0.176), traffic source (0.167) and visitor type (0.103) are clearly associated with purchasing. Browser and region are not significant at the 5% level (p = 0.135 and 0.288), and the weekend effect is statistically significant but negligible in size (V = 0.027). Sparse levels were pooled first so that every expected cell count exceeded 5.

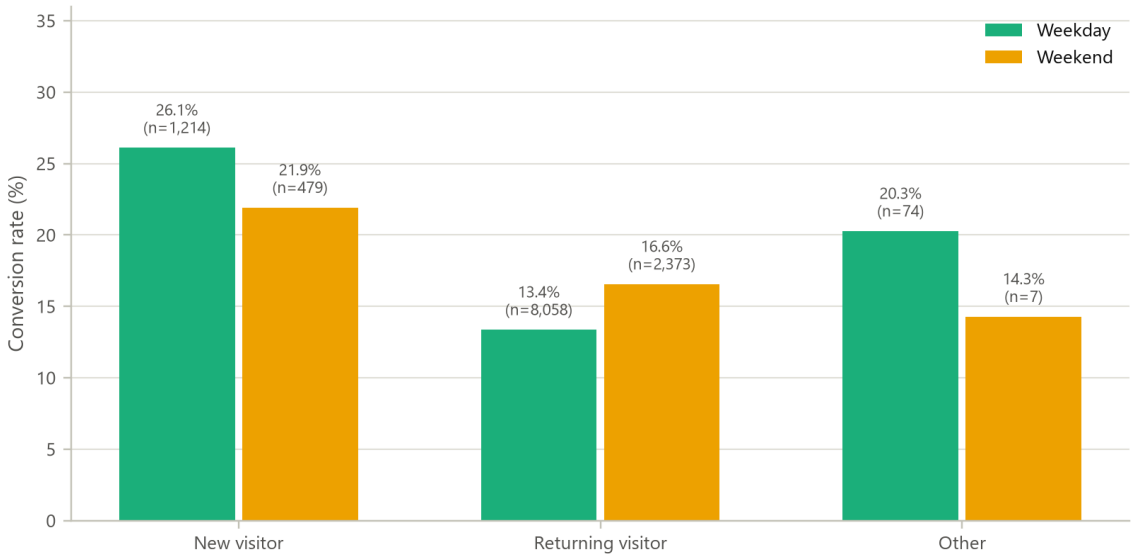


Figure 4.10: Conversion rate by visitor type and day type (three-way contingency)

What it shows. A grouped bar chart of conversion rate for each visitor type, split into weekday and weekend sessions. This is a three-variable contingency table drawn as a chart.

Why it was created. Introducing a third variable tests whether a two-way relationship holds in every subgroup, which guards against drawing conclusions from a confounded comparison.

Insights obtained. New visitors convert more often than returning visitors (24.9% against 14.1%), which seems counter-intuitive. The third variable shows an interaction: new visitors buy more on weekdays (26.1% against 21.9%), while returning visitors buy more at weekends (16.6% against 13.4%). The new-visitor advantage survives when PageValues is held fixed (with zero page value: 7.1% for new and 3.4% for returning visitors), so it is not explained by page value alone. A plausible explanation is that many returning visitors come back to compare or re-check items without a firm plan to buy.

4.12 Multivariate View: Pair Plot

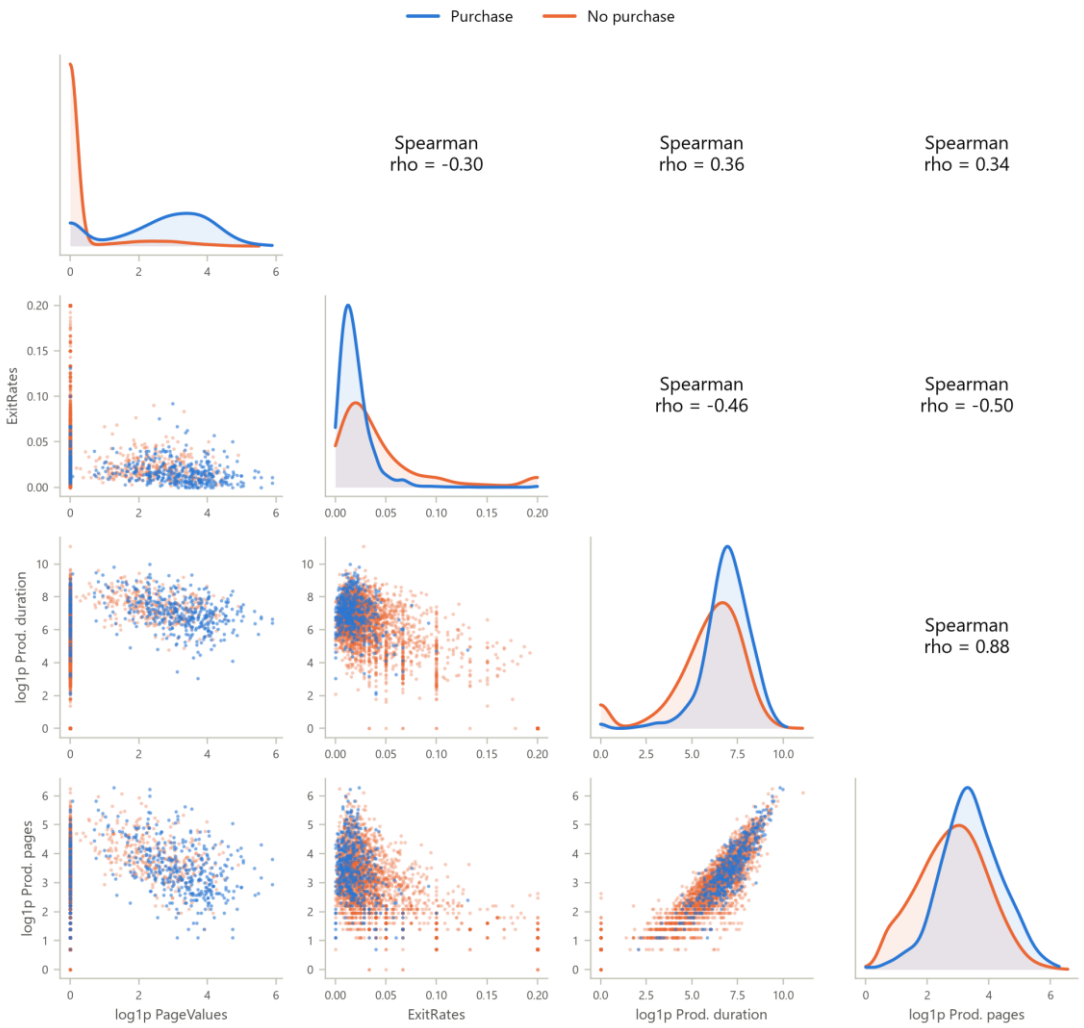


Figure 4.11: Pair plot of four key features coloured by class (3,000-session random sample)

What it shows. A 4 x 4 matrix with class-specific density curves on the diagonal, scatterplots below it and Spearman coefficients above it, for page value, exit rate, product time and product pages.

Why it was created. A pair plot shows every bivariate relationship among the strongest features at once and shows whether the classes separate in any pair of dimensions.

Insights obtained. No pair separates the classes completely, but page value and exit rate together come close. Buyers concentrate at positive page values and exit rates below 0.05, while non-buyers fill the zero-page-value column at every exit rate. The diagonal densities repeat the univariate story: buyers browse longer and visit more product pages. Product time and product pages are strongly collinear (rho = 0.88).

4.13 Multivariate View: Principal Component Analysis

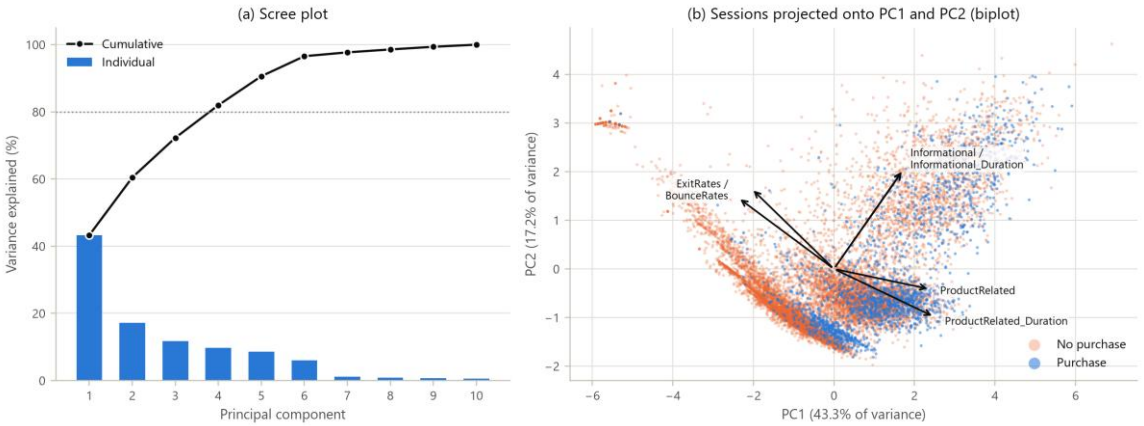


Figure 4.12: Scree plot and PCA biplot of the ten log-transformed numerical features

What it shows. Panel (a) shows the variance explained by each principal component. Panel (b) projects every session onto the first two components, colours it by class and overlays the loading arrows of the most influential variables.

Why it was created. PCA summarises correlated variables into a few uncorrelated axes. This tests whether the behavioural variables measure a small number of underlying ideas, and whether those ideas separate buyers from non-buyers.

Insights obtained. The first component explains 43.3% of the variance and the second 17.2%. Four components are needed to pass 80%. PC1 is an engagement axis (product time, product pages and administrative pages load positively, exit rate negatively). PC2 separates product-focused browsing (negative scores) from informational browsing and bouncing (positive scores). Buyers drift towards high PC1, but the two classes overlap heavily in this unsupervised view, because PCA ignores the label and gives little weight to PageValues. This explains why a supervised model is still needed. The small isolated cluster at the upper left consists of single-page bounce sessions.

4.14 Interact: Exploratory Dashboard

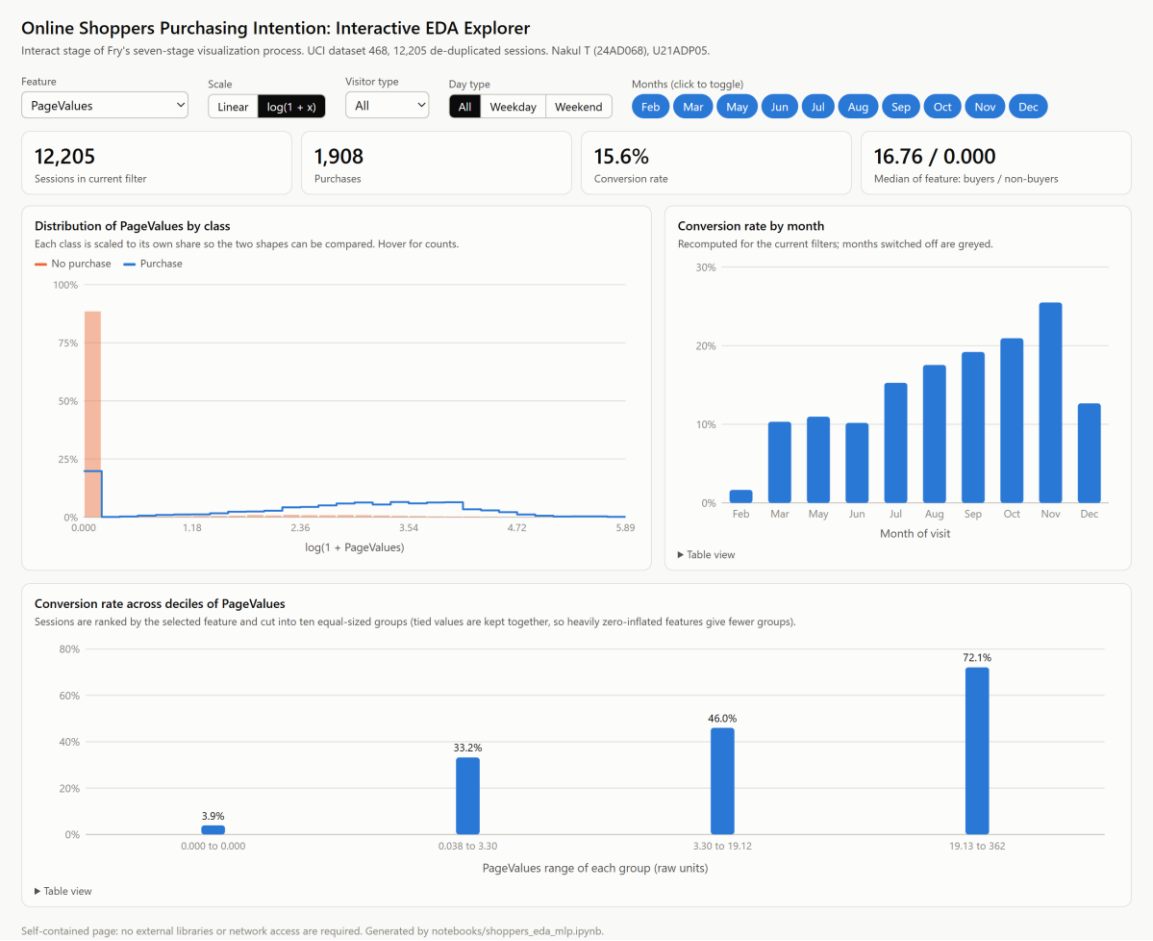


Figure 4.13: Interactive EDA dashboard (dashboard/index.html) showing the PageValues view

What it shows. A self-contained HTML page that recomputes the class-wise distribution of any numerical feature, the monthly conversion rate and a decile lift chart whenever the reader changes the feature, the scale (linear or log1p), the visitor type, the day type or the set of months. Hovering over any bar shows exact counts, and a table view is available beneath each chart.

Why it was created. Interaction is the seventh of Fry's stages. Static figures answer the questions the analyst chose, whereas the dashboard lets a manager ask new ones, such as whether returning visitors in November behave like new visitors in May. It runs offline in any browser and needs no installation.

Insights obtained. Filtering shows which patterns are stable. The PageValues gap between buyers and non-buyers and the falling conversion across exit-rate quintiles appear in every visitor-type and day-type subset with enough sessions. The November peak, however, belongs to returning visitors: new visitors convert best in August on weekdays and in September at weekends. Switching to the linear scale also shows why the log scale was needed: on the raw axis almost every session falls into the first bin.

4.15 Summary of Statistical Findings

Table 4.6: Summary of the statistical and distribution-based analysis

Analysis	Method	Key result
Univariate	Five-number summaries, skewness, histograms, ladder of powers	All behavioural variables right-skewed (1.9 to 7.5); log is the symmetrising rung
Inequality	Lorenz curve, Gini	Top 10% of sessions hold 45% of browsing time and 82% of page value
Discrete distribution	Poisson vs negative binomial (MLE)	Dispersion index 62; negative binomial strongly preferred
Continuous distribution	Weibull, lognormal, gamma, exponential (MLE, AIC, KS)	Weibull best, shape 0.81 (decreasing hazard)
Binomial proportion	Wilson intervals, chi-square homogeneity	Conversion 1.7% (Feb) to 25.5% (Nov); $p < 10^{-74}$
Bivariate	Mann-Whitney U, chi-square, Cramer's V, Spearman	PageValues AUC 0.86; Month V 0.18; Browser and Region not significant
Third variable	Three-way contingency table	Visitor-type effect reverses between weekday and weekend
Multivariate	Pair plot, PCA	4 components for 80% variance; classes overlap without supervision

5. DEEP LEARNING MODEL

5.1 Choice of Model

Each session is a fixed-length vector of 62 engineered features with no temporal order inside the record, so a fully connected feed-forward network, the multilayer perceptron (MLP), is the appropriate deep learning architecture. Convolutional and recurrent networks would need grid-shaped or sequential inputs, which the aggregated data do not provide. The MLP also matches the model that performed best in the original study of this dataset [1], which makes the results comparable. Each hidden unit computes $a = \mathrm{ReLU}(w^{T}x + b)$, and the output unit applies the sigmoid function to produce a purchase probability.

5.2 Architecture

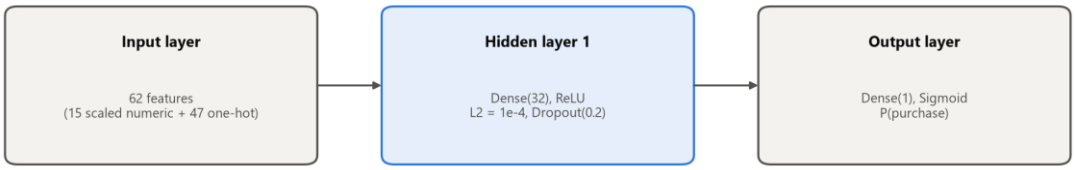


Figure 5.1: Architecture of the selected multilayer perceptron

Table 5.1: Layer-by-layer summary of the selected network

Layer	Output shape	Activation / setting	Parameters
Input (session_features)	62	-	0
Dense 1	32	ReLU, L2 penalty 1e-4	2,016
Dropout 1	32	rate 0.2	0
Output (purchase_probability)	1	Sigmoid	33
Total			2,049 trainable

Table 5.1 shows how compact the selected network is: a single hidden layer of 32 ReLU units with dropout, 2,049 trainable parameters in total. Section 5.4 explains why a larger network was not chosen.

5.3 Training Parameters

Table 5.2: Training configuration

Setting	Value
Framework	TensorFlow 2.21.0 / Keras 3.15.1 (deterministic operations, seed 42)
Loss function	Binary cross-entropy with class weights 0.593 (no purchase) and 3.197 (purchase)
Optimiser	Adam [8], initial learning rate selected by tuning (1e-3)
Batch size / maximum epochs	128 / 150
Early stopping	Monitor validation ROC-AUC, patience 15 epochs, best weights restored
Learning-rate schedule	ReduceLROnPlateau on validation loss: factor 0.5, patience 6, minimum 1e-5
Regularisation	Dropout [9] after every hidden layer; L2 weight penalty 1e-4
Metrics tracked	Loss, accuracy (threshold 0.5), ROC-AUC, PR-AUC
Decision threshold	Chosen on the validation set to maximise F1 (0.648)

5.4 Hyperparameter Tuning

A basic grid search covered three architectures (32; 64-32; 128-64-32 hidden units), two dropout rates (0.2 and 0.4) and two learning rates (1e-3 and 3e-4), giving twelve configurations. Each was trained with the same seed and early-stopping rule and scored on validation ROC-AUC. The search took 182 s on a CPU.

Table 5.3: Hyperparameter search results (selected configuration: #1)

#	Hidden units	Dropout	Learning rate	Epochs	Best epoch	Val ROC-AUC	Val PR-AUC	Params
1	32	0.2	0.001	46	31	0.9342	0.7415	2,049
2	32	0.2	0.0003	116	101	0.9331	0.7395	2,049
3	32	0.4	0.001	60	45	0.9338	0.7389	2,049
4	32	0.4	0.0003	113	98	0.9335	0.7405	2,049
5	64-32	0.2	0.001	32	17	0.9318	0.7321	6,145
6	64-32	0.2	0.0003	48	33	0.9317	0.7375	6,145
7	64-32	0.4	0.001	40	25	0.9319	0.7294	6,145
8	64-32	0.4	0.0003	53	38	0.9317	0.7351	6,145
9	128-64-32	0.2	0.001	26	11	0.9321	0.7371	18,433
10	128-64-32	0.2	0.0003	46	31	0.9327	0.7386	18,433
11	128-64-32	0.4	0.001	28	13	0.9339	0.7370	18,433
12	128-64-32	0.4	0.0003	61	46	0.9326	0.7359	18,433

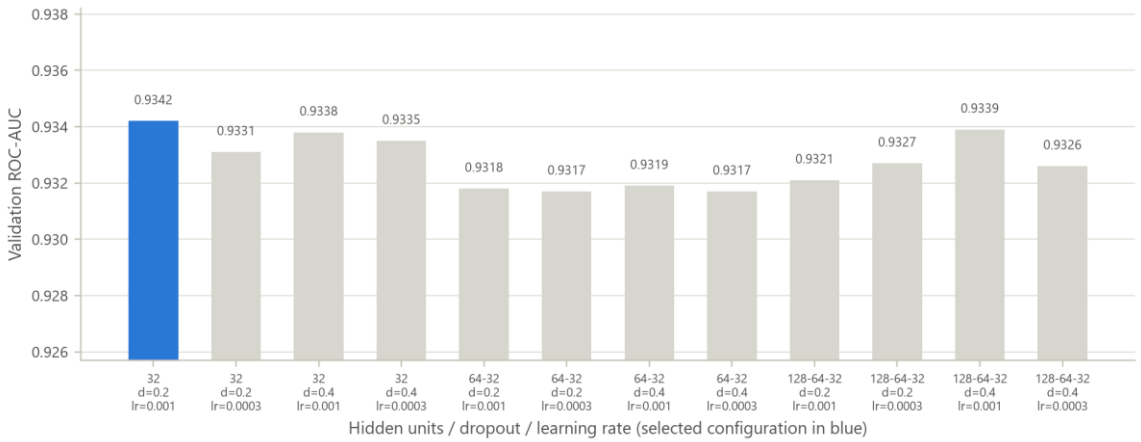


Figure 5.2: Validation ROC-AUC of the twelve hyperparameter configurations

What it shows. One bar per configuration, labelled with its hidden units, dropout and learning rate; the selected configuration is highlighted.

Why it was created. The chart shows how sensitive the model is to its hyperparameters, which matters as much as the single best score.

Insights obtained. Every configuration lies between 0.9317 and 0.9342, a spread of only 0.0025. Larger networks with up to 18,433 parameters bring no gain over the 2,049-parameter network, and they reach their best epoch sooner and then overfit. The task is close to separable once page value and exit behaviour are known, so the smallest network was preferred.

5.5 Feature Analysis and Importance

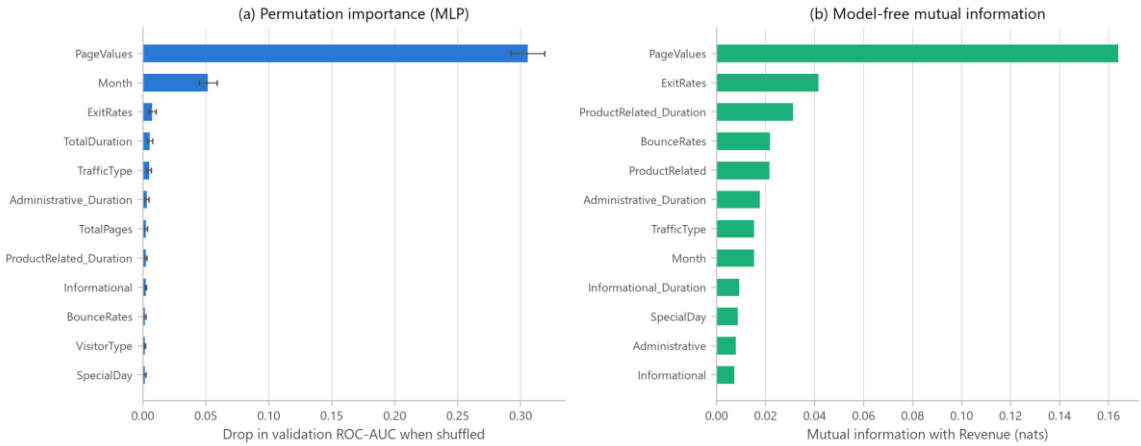


Figure 5.3: Permutation importance of the trained MLP and model-free mutual information

What it shows. Panel (a) shows how much the validation ROC-AUC drops when each original field (all of its one-hot columns together) is randomly shuffled, averaged over ten repetitions [14]. Panel (b) ranks the raw fields by their mutual information with the target, a measure that uses no model.

Why it was created. Permutation importance shows what the trained network actually relies on. Mutual information shows which variables carry information about the target at all. Agreement between the two increases confidence, and differences between them are informative too.

Insights obtained. Shuffling PageValues (together with its HasPageValue flag) lowers ROC-AUC by 0.306, about six times more than any other field. Month is second (0.052), consistent with the seasonality in Section 4.6, followed by ExitRates and the engineered TotalDuration. Region, Informational_Duration and Administrative contribute nothing, as the chi-square tests predicted. The mutual-information ranking agrees at the top (PageValues 0.164 nats, ExitRates 0.042) but places Month lower. Month matters to the network mainly in combination with the other inputs.

5.6 Baseline for Comparison

A model's value only shows against a simpler alternative. Logistic regression with balanced class weights was trained on exactly the same 62 inputs and splits, and its threshold was also tuned on validation data (0.591). Any advantage of the MLP must therefore come from non-linear interactions, not from better preprocessing.

6. RESULT VISUALIZATION AND INTERPRETATION

All results below were computed on the 1,831 test sessions, which played no part in training, tuning or threshold selection. The test set keeps the 15.6% purchase rate of the full data.

6.1 Loss versus Epoch

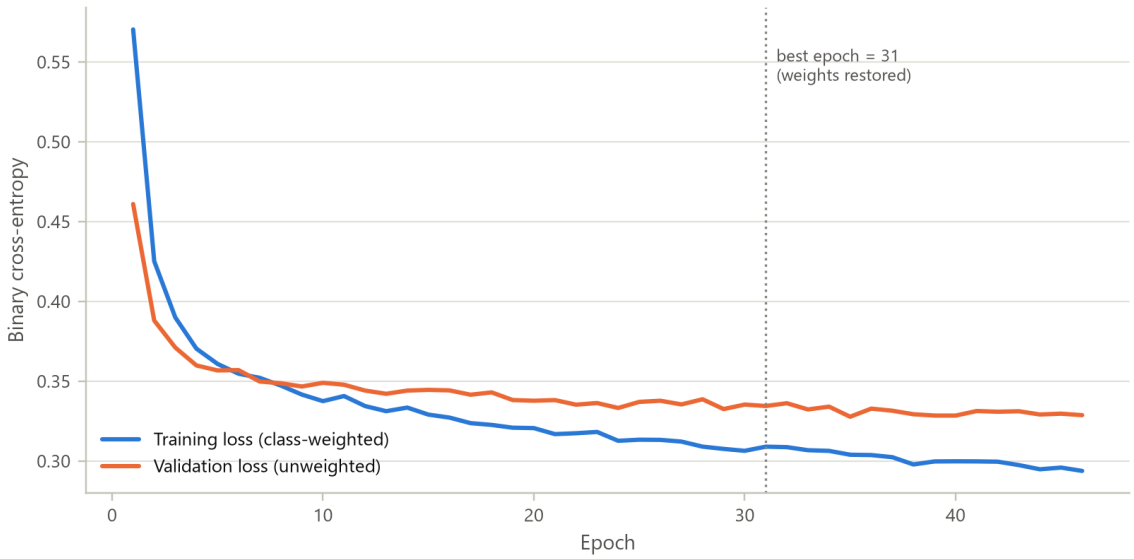


Figure 6.1: Training and validation loss of the selected network

What it shows. Binary cross-entropy for each epoch on the training data (class-weighted) and the validation data (unweighted), with the epoch whose weights were restored marked.

Why it was created. Learning curves are the first diagnostic of any network. They show whether it learned, whether it converged and whether it began to overfit.

Insights obtained. Both losses fall steeply over the first five epochs and then flatten. Validation loss reaches about 0.334 at epoch 31, the epoch whose weights were restored. Training loss keeps falling slowly (to 0.294 by epoch 46) while validation loss stays flat, which is the onset of overfitting that early stopping guards against. The validation curve is not inflated by class weights, so it lies on a different scale from the training curve and only its shape should be compared.

6.2 Accuracy and ROC-AUC versus Epoch

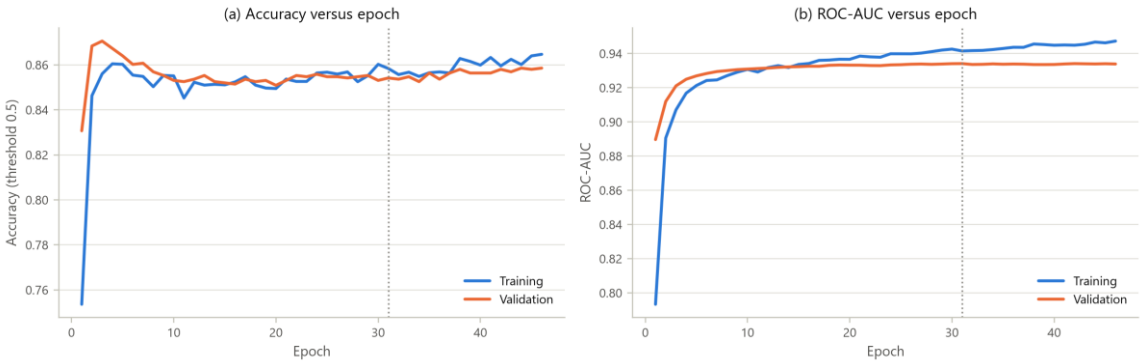


Figure 6.2: Accuracy and ROC-AUC per epoch for training and validation data

What it shows. Panel (a) tracks accuracy at the default 0.5 threshold and panel (b) tracks ROC-AUC, for both training and validation data.

Why it was created. Accuracy is the metric most readers expect, but under imbalance it can mislead. Showing ROC-AUC alongside it makes the training dynamics clearer.

Insights obtained. Validation accuracy settles near 85.4% while validation ROC-AUC climbs to 0.934. The gap between training and validation AUC stays small (0.941 against 0.934 at the restored epoch), so the model generalises rather than memorises. Validation accuracy actually peaks in the first epochs and then dips, because the class weights push the network to flag more potential buyers, which improves recall at the cost of raw accuracy.

6.3 Confusion Matrix

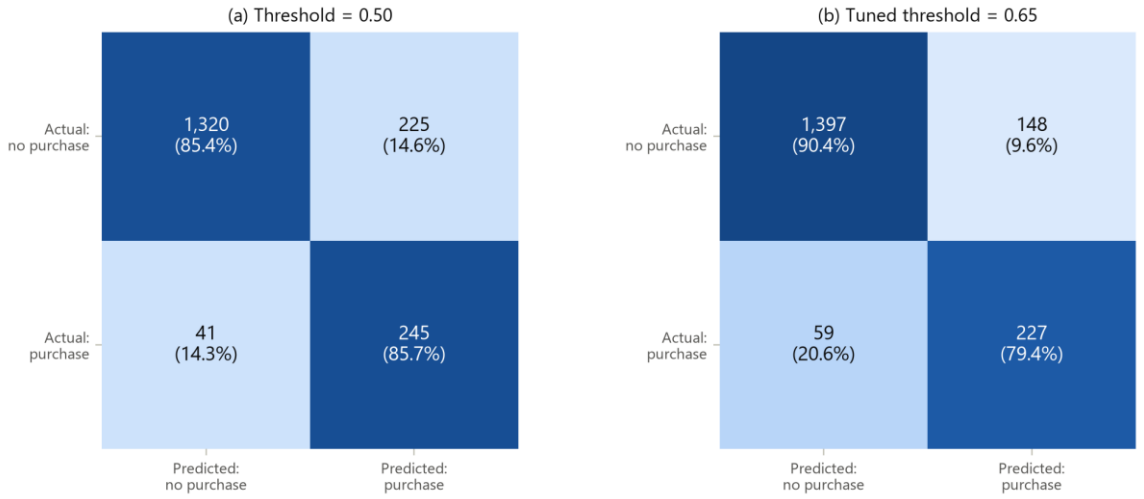


Figure 6.3: Test-set confusion matrices at the default and the validation-tuned threshold

What it shows. Counts and row percentages of test predictions at the default threshold of 0.5 and at the threshold of 0.65 chosen on validation data.

Why it was created. The confusion matrix breaks overall accuracy into the four outcomes that carry different business costs: a correctly flagged buyer, a missed buyer, a false alarm and a correctly ignored visitor.

Insights obtained. At 0.5 the network finds 245 of 286 buyers (recall 85.7%) but raises 225 false alarms. Moving the threshold to 0.65 gives up a little recall (79.4%, 227 buyers found) and cuts false alarms by a third (to 148). Precision rises from 52.1% to 60.5% and accuracy from 85.5% to 88.7%. Which threshold is better depends on the cost of a missed buyer compared with the cost of an unnecessary offer.

6.4 ROC Curve and AUC

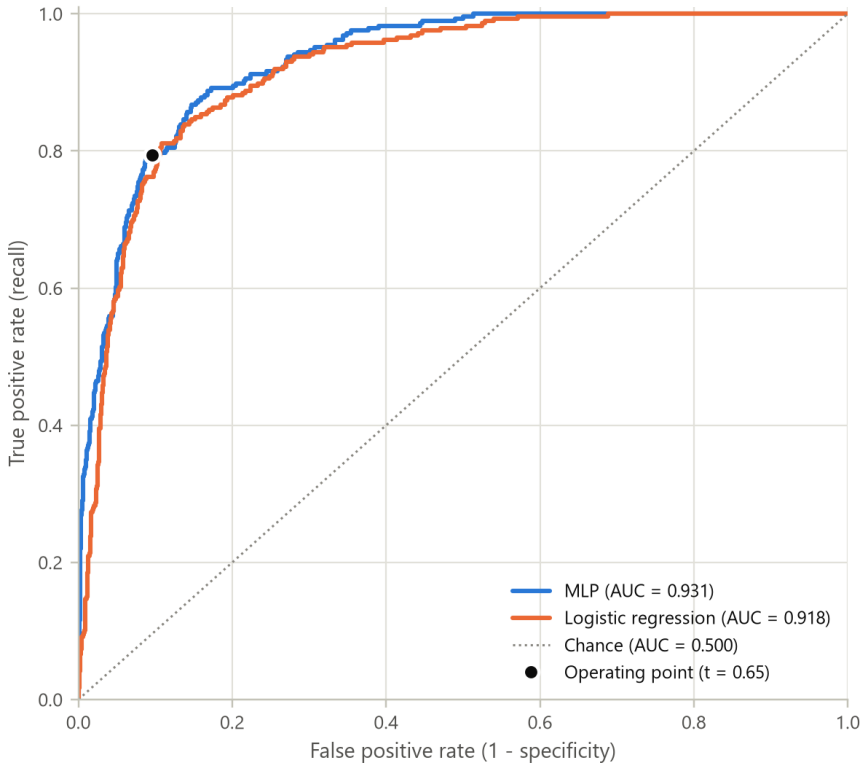


Figure 6.4: ROC curves of the MLP and the logistic-regression baseline on the test set

What it shows. The true-positive rate against the false-positive rate for every possible threshold, for the MLP and the baseline, with the chance diagonal and the MLP's chosen operating point.

Why it was created. ROC analysis evaluates a classifier independently of any single threshold [11]. The area under the curve equals the probability that a random buyer receives a higher score than a random non-buyer.

Insights obtained. The MLP achieves a test ROC-AUC of **0.931** against 0.918 for logistic regression. Its curve lies above the baseline mainly in the low false-positive region, which is where a retailer would operate. At the chosen threshold it detects 79% of buyers while wrongly flagging 10% of non-buyers.

6.5 Precision-Recall Curve and Threshold Analysis

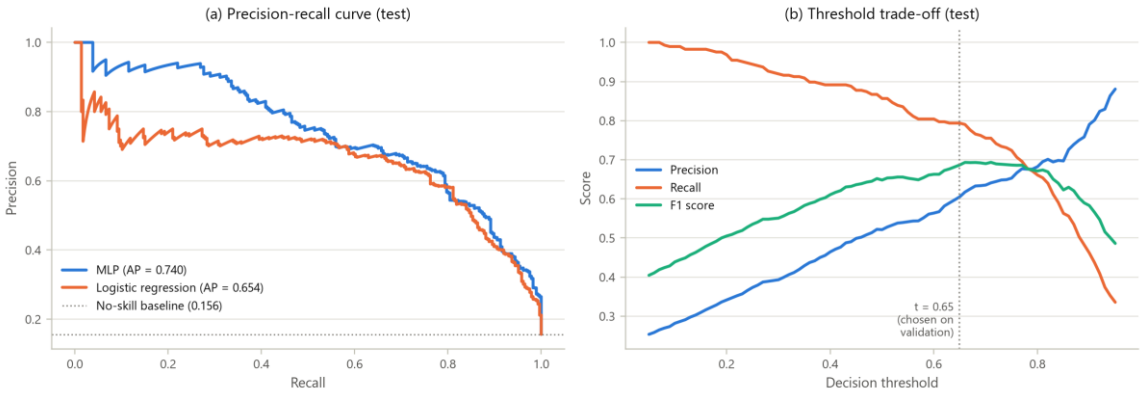


Figure 6.5: Precision-recall curves and precision, recall and F1 as functions of the threshold

What it shows. Panel (a) shows precision against recall for both models, with the no-skill baseline equal to the 15.6% purchase rate. Panel (b) shows how precision, recall and F1 change as the decision threshold moves.

Why it was created. With only 15.6% positives, precision-recall curves describe minority-class performance more honestly than ROC curves [10]. The threshold plot explains the operating point used in Section 6.3.

Insights obtained. The MLP's average precision (PR-AUC) is **0.740**, compared with 0.654 for logistic regression and 0.156 for random guessing. The gain over the baseline is much larger here than on the ROC curve, because the MLP holds precision above 0.9 up to about 30% recall: the sessions it is most confident about really are buyers. In panel (b), F1 is flat between thresholds of about 0.6 and 0.8, so the chosen value of 0.65 is not a fragile optimum.

6.6 Score Distribution and Error Analysis

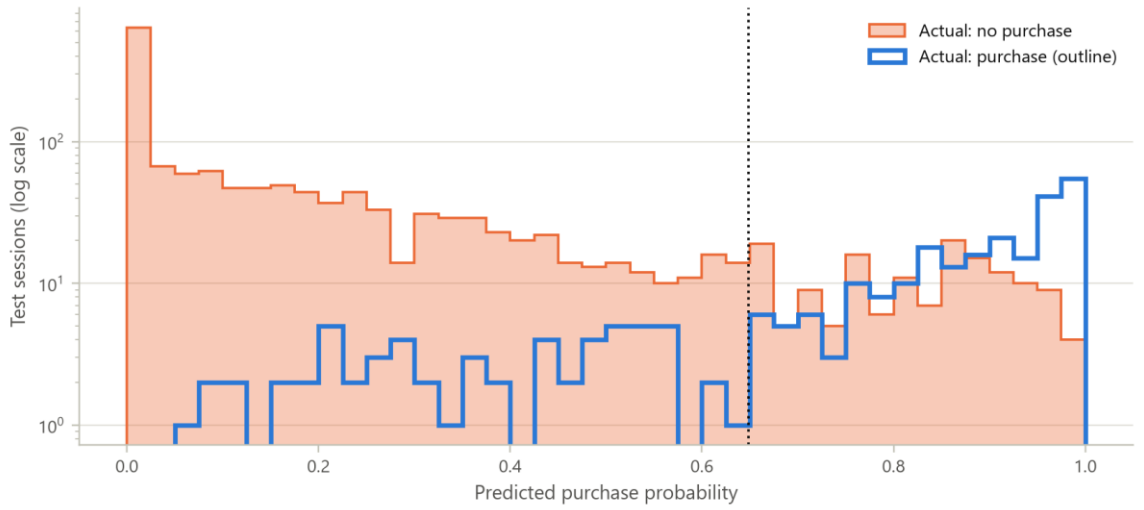


Figure 6.6: Distribution of predicted purchase probability by true class (test set, log scale)

What it shows. Histograms of the predicted probability for actual buyers and actual non-buyers, with the decision threshold marked.

Why it was created. For a classifier the analogue of an error-distribution chart is the separation of scores by class. It shows where the errors come from rather than only how many there are.

Insights obtained. Non-buyers pile up near zero (median score 0.078) and buyers near one (median 0.881), so the two classes are well separated. The errors sit in the middle range. Of the buyers the model misses, 83% had a page value of zero. These are shoppers who bought without passing through the pages that normally signal intent, and no session-level feature in the dataset can reveal them.

6.7 Summary of Test Performance

Table 6.1: Test-set performance of the MLP and the logistic-regression baseline

Metric	MLP (t = 0.50)	MLP (t = 0.65)	LogReg (t = 0.50)	LogReg (t = 0.59)
Accuracy	0.8547	0.8869	0.8624	0.8793
Balanced accuracy	0.8555	0.8490	0.8444	0.8473
Precision	0.5213	0.6053	0.5392	0.5827
Recall (sensitivity)	0.8566	0.7937	0.8182	0.8007
Specificity	0.8544	0.9042	0.8706	0.8939
F1 score	0.6481	0.6868	0.6500	0.6745
Matthews corr.	0.5909	0.6278	0.5880	0.6142
ROC-AUC	0.9314	0.9314	0.9184	0.9184
PR-AUC (average precision)	0.7400	0.7400	0.6542	0.6542

The MLP is better on every threshold-free metric: ROC-AUC improves by 0.013 and PR-AUC by 0.086. At tuned thresholds it also wins on accuracy, precision, F1 and MCC (F1 0.687 against 0.674). The size of the gain is modest, which is consistent with the exploratory finding that a single feature, PageValues, carries most of the signal. The extra capacity of the network is spent on the subtler interactions, such as month with visitor type and exit behaviour with engagement, that a linear model cannot represent.

7. CONCLUSION AND FUTURE SCOPE

7.1 What the Data Revealed

- Online sessions are extremely unequal: a tenth of sessions account for 45% of product-browsing time and 82% of page value, and every behavioural variable is right-skewed or zero-inflated.
- Page counts are over-dispersed (dispersion index 62), so a negative binomial describes them far better than a Poisson. Browsing time follows a Weibull law with shape 0.81, meaning engaged sessions tend to stay engaged.
- Conversion is a binomial process whose rate varies strongly by month, from 1.7% in February to 25.5% in November. Volume and conversion peak in different months.
- Where a visitor goes matters more than how much they browse. PageValues (AUC 0.86) and ExitRates are the dominant signals, while page counts correlate only weakly with purchase.
- A third variable changes the picture: new visitors out-convert returning visitors on weekdays, while returning visitors convert more at weekends.

7.2 How the Model Performed

The tuned multilayer perceptron reached a test ROC-AUC of 0.931, a PR-AUC of 0.740, an accuracy of 88.7% and an F1 score of 0.687. It identified 79% of buyers while flagging only 10% of non-buyers, and it beat a class-weighted logistic regression on every threshold-free metric. Permutation importance confirmed that the network learned the same drivers that the exploratory analysis identified, so the model and the EDA support each other.

7.3 Challenges Faced

- Class imbalance (15.6% buyers) made accuracy misleading. It was handled with class weights, threshold tuning and precision-recall evaluation.
- Duplicates and structural zeros looked like data errors at first. Recognising them as bounce sessions and analytics artefacts required careful profiling.
- Classical outlier rules failed on zero-inflated variables, so transformation had to replace deletion.
- Strong collinearity between counts and durations, and between bounce and exit rates, meant that importance had to be measured for groups of related features rather than for individual columns.
- Two months are missing entirely (January and April), so seasonal conclusions cover ten months only.

7.4 Recommendations for the Retailer

- Trigger real-time offers or live chat when a session reaches a page with positive page value but shows a rising exit rate. These visitors are close to buying and at risk of leaving.
- Plan campaigns around conversion as well as traffic: November and October convert best, while May attracts the most visits but converts at only 11%.
- Investigate why returning weekday visitors convert less. Saved-cart reminders or comparison tools may help visitors who return to research rather than to buy.
- Do not rely on special-day proximity alone; sessions close to special days convert slightly less often.

7.5 Future Scope

- Model the page-by-page clickstream with a recurrent network, as in [1], to predict abandonment during a session rather than after it.
- Add cost-sensitive thresholding based on the actual margin of a sale and the cost of an incentive, instead of maximising F1.
- Probe the network with SHAP values or partial-dependence plots to explain individual predictions to marketing teams.
- Calibrate the predicted probabilities (for example with isotonic regression) so they can be used directly as purchase likelihoods.
- Validate on data from other months and years, particularly January and April, and monitor for concept drift.

REFERENCES

[1] C. O. Sakar, S. O. Polat, M. Katircioglu, and Y. Kastro, "Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks," *Neural Computing and Applications*, vol. 31, no. 10, pp. 6893–6908, 2019, doi: 10.1007/s00521-018-3523-0.

[2] C. O. Sakar and Y. Kastro, "Online Shoppers Purchasing Intention Dataset," UCI Machine Learning Repository, 2018. [Online]. Available: https://doi.org/10.24432/C5F88Q

[3] B. Fry, *Visualizing Data: Exploring and Explaining Data with the Processing Environment*. Sebastopol, CA, USA: O'Reilly Media, 2008.

[4] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[5] W. W. Moe and P. S. Fader, "Dynamic conversion behavior at e-commerce sites," *Management Science*, vol. 50, no. 3, pp. 326–335, 2004.

[6] R. E. Bucklin and C. Sismeiro, "A model of web site browsing behavior estimated on clickstream data," *Journal of Marketing Research*, vol. 40, no. 3, pp. 249–267, 2003.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[10] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.

[11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[12] A. C. Cameron and P. K. Trivedi, *Regression Analysis of Count Data*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2013.

[13] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

APPENDIX A: PROJECT REPOSITORY AND REPRODUCIBILITY

The complete project, including this report, the executed notebook, the dataset, all figures and the interactive dashboard, is maintained in the GitHub repository **https://github.com/nakultt/online-shoppers-eda-mlp**. Its structure is:

```
online-shoppers-eda-mlp/
|-- README.md                       project summary, results and how to run
|-- requirements.txt                Python dependencies
|-- data/online_shoppers_intention.csv   UCI dataset 468 (CC BY 4.0)
|-- notebooks/shoppers_eda_mlp.ipynb     executed notebook (all outputs)
|-- src/shoppers_eda_mlp.py         same code as a script (jupytext pair)
|-- src/dashboard_template.html     template of the interactive dashboard
|-- dashboard/index.html            self-contained interactive dashboard
|-- figures/                        25 figures used in this report
|-- results/                        results.json, tables (CSV), best_mlp.keras
`-- report/                         this report (DOCX and PDF)
```

To reproduce every number in this report, install the requirements and run the notebook from top to bottom (about five minutes on a laptop CPU):

```
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace notebooks/shoppers_eda_mlp.ipynb
```

Seeds are fixed (42) and TensorFlow's deterministic operations are enabled, so the tuning table, the selected configuration and all test metrics reproduce exactly.

APPENDIX B: SOURCE CODE (KEY SECTIONS)

The listings below are taken verbatim from `src/shoppers_eda_mlp.py`. Plotting code is omitted for brevity; it is available in the repository.

B.1 Data-quality audit and de-duplication

```
audit = {"explicit_missing": int(raw.isna().sum().sum())}

# Implicit problems: a page count and its duration should agree with each other
logic_rows = []
for cnt, dur in [("Administrative", "Administrative_Duration"),
                 ("Informational", "Informational_Duration"),
                 ("ProductRelated", "ProductRelated_Duration")]:
    logic_rows.append({
        "Pair": f"{cnt} / {dur}",
        "count = 0 but duration > 0": int(((raw[cnt] == 0) & (raw[dur] > 0)).sum()),
        "count > 0 but duration = 0": int(((raw[cnt] > 0) & (raw[dur] == 0)).sum()),
    })
logic = pd.DataFrame(logic_rows)
audit["logic"] = logic.to_dict(orient="records")
audit["rates_within_0_0.2"] = bool(raw["BounceRates"].between(0, 0.2).all() and raw["ExitRates"].between(0,
0.2).all())
audit["negative_values"] = int((raw[NUM] < 0).sum().sum())

# Exact duplicate rows
dup_mask = raw.duplicated(keep="first")
dups = raw[dup_mask]
audit["duplicates"] = int(dup_mask.sum())
audit["dup_single_product_page_pct"] = r(100 * (dups["ProductRelated"] <= 1).mean(), 1)
audit["dup_bounce_0.2_pct"] = r(100 * (dups["BounceRates"] == 0.2).mean(), 1)
audit["dup_zero_duration_pct"] = r(100 * (dups["ProductRelated_Duration"] == 0).mean(), 1)
audit["dup_revenue_pct"] = r(100 * dups[TARGET].mean(), 1)

df = raw.drop_duplicates().reset_index(drop=True)
audit["rows_after_dedup"] = int(len(df))
audit["purchase_pct_after_dedup"] = r(100 * df[TARGET].mean(), 2)
print(json.dumps(audit, indent=1))
logic
```

B.2 Maximum-likelihood negative binomial fit

```
def fit_negbin(x):
    """Maximum-likelihood negative binomial fit (scipy parameterisation n=r, p)."""
    x = np.asarray(x)
    m, v = x.mean(), x.var()
    r0 = m ** 2 / max(v - m, 1e-6)
    p0 = r0 / (r0 + m)

    def nll(theta):
        rr, pp = np.exp(theta[0]), 1 / (1 + np.exp(-theta[1]))
        return -stats.nbinom.logpmf(x, rr, pp).sum()

    res = optimize.minimize(nll, [np.log(r0), np.log(p0 / (1 - p0))], method="Nelder-Mead",
                            options={"xatol": 1e-8, "fatol": 1e-8, "maxiter": 4000})
    return np.exp(res.x[0]), 1 / (1 + np.exp(-res.x[1])), -res.fun
```

B.3 Continuous distribution fitting

```
cands = {"Lognormal": stats.lognorm, "Gamma": stats.gamma, "Weibull": stats.weibull_min, "Exponential":
stats.expon}
cont = {}
for name, dist in cands.items():
    params = dist.fit(y_dur, floc=0)
    ll = dist.logpdf(y_dur, *params).sum()
    k = len(params) - 1
    ks = stats.kstest(y_dur, dist.cdf, args=params)
    cont[name] = {"params": [float(p) for p in params], "loglik": r(ll, 1), "aic": r(2 * k - 2 * ll, 1),
                  "ks_D": r(ks.statistic, 4), "ks_p": float(ks.pvalue)}
```

B.4 Lorenz curve and Gini coefficient

```
def lorenz_curve(x):
    """Return the Lorenz curve (population share, value share) and the Gini coefficient."""
    x = np.sort(np.asarray(x, dtype=float))
    cum = np.insert(np.cumsum(x) / x.sum(), 0, 0.0)
    pop = np.linspace(0, 1, len(cum))
    gini = 1 - 2 * np.trapezoid(cum, pop)
    return pop, cum, gini
```

B.5 Wilson score interval

```
def wilson(k, n, z=1.96):
    """Wilson score confidence interval for a binomial proportion."""
    p = k / n
    den = 1 + z ** 2 / n
    centre = (p + z ** 2 / (2 * n)) / den
    half = z * np.sqrt(p * (1 - p) / n + z ** 2 / (4 * n ** 2)) / den
    return centre - half, centre + half
```

B.6 Tukey's resistant line

```
def resistant_line(x, y, iters=20):
    """Tukey's three-group resistant line with iterative polishing of the slope."""
    t1, t2 = np.quantile(x, [1 / 3, 2 / 3])
    L, R = x <= t1, x >= t2
    M = ~L & ~R
    xL, xR = np.median(x[L]), np.median(x[R])
    b = (np.median(y[R]) - np.median(y[L])) / (xR - xL)
    for _ in range(iters):
        res = y - b * x
        db = (np.median(res[R]) - np.median(res[L])) / (xR - xL)
        b += db
        if abs(db) < 1e-7:
            break
    res = y - b * x
    a = (np.median(res[L]) + np.median(res[M]) + np.median(res[R])) / 3
    return a, b
```

B.7 Feature engineering, encoding, scaling and split

```
mdl = df.copy()
mdl["Month"] = mdl["Month"].replace({"June": "Jun"})
mdl["TotalPages"] = mdl[["Administrative", "Informational", "ProductRelated"]].sum(axis=1)
mdl["TotalDuration"] = mdl[["Administrative_Duration", "Informational_Duration",
"ProductRelated_Duration"]].sum(axis=1)
mdl["DurationPerProductPage"] = mdl["ProductRelated_Duration"] / mdl["ProductRelated"].clip(lower=1)
mdl["HasPageValue"] = (mdl["PageValues"] > 0).astype(int)
mdl["Weekend"] = mdl["Weekend"].astype(int)
LOG_COLS = ["Administrative", "Administrative_Duration", "Informational", "Informational_Duration",
            "ProductRelated", "ProductRelated_Duration", "PageValues", "TotalPages", "TotalDuration",
            "DurationPerProductPage"]
for c in LOG_COLS:
    mdl[c] = np.log1p(mdl[c])                       # compress heavy right tails
y_all = mdl.pop(TARGET).astype(int).to_numpy()

# Stratified 70 / 15 / 15 split so that every subset keeps the 15.6% purchase rate
idx = np.arange(len(mdl))
idx_tr, idx_tmp = train_test_split(idx, test_size=0.30, stratify=y_all, random_state=SEED)
idx_va, idx_te = train_test_split(idx_tmp, test_size=0.50, stratify=y_all[idx_tmp], random_state=SEED)

CAT_M = ["Month", "OperatingSystems", "Browser", "Region", "TrafficType", "VisitorType"]
for c in CAT_M:                                      # pool levels rarer than 1% (learned on train only)
    s = mdl[c].astype(str)
    keep = set(s.iloc[idx_tr].value_counts(normalize=True).loc[lambda v: v >= 0.01].index)
    mdl[c] = s.where(s.isin(keep), "Other")
NUM_M = LOG_COLS + ["BounceRates", "ExitRates", "SpecialDay", "Weekend", "HasPageValue"]
pre = ColumnTransformer([("num", StandardScaler(), NUM_M),
                         ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CAT_M)])
X_tr = pre.fit_transform(mdl.iloc[idx_tr]).astype("float32")
X_va = pre.transform(mdl.iloc[idx_va]).astype("float32")
X_te = pre.transform(mdl.iloc[idx_te]).astype("float32")
```

```
y_tr, y_va, y_te = y_all[idx_tr], y_all[idx_va], y_all[idx_te]
feat_names = list(pre.get_feature_names_out())
groups = {}
for k, f in enumerate(feat_names):
    base = f.split("__", 1)[1]
    orig = next((c for c in CAT_M if base.startswith(c + "_")), base) if f.startswith("cat__") else base
    if orig == "HasPageValue":
        orig = "PageValues"                        # the flag is derived from PageValues: one information source
    groups.setdefault(orig, []).append(k)
```

B.8 MLP definition and hyperparameter tuning

```
def build_mlp(input_dim, hidden=(64, 32), dropout=0.3, lr=1e-3, l2=1e-4):
    """Fully connected network: [Dense -> ReLU -> Dropout] x len(hidden) -> Dense(1, sigmoid)."""
    inp = keras.Input(shape=(input_dim,), name="session_features")
    x = inp
    for i, units in enumerate(hidden):
        x = layers.Dense(units, activation="relu", kernel_regularizer=regularizers.l2(l2), name=f"dense_{i +
1}")(x)
        x = layers.Dropout(dropout, name=f"dropout_{i + 1}")(x)
    out = layers.Dense(1, activation="sigmoid", name="purchase_probability")(x)
    model = keras.Model(inp, out, name="purchase_mlp")
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr), loss="binary_crossentropy",
                  metrics=[keras.metrics.BinaryAccuracy(name="accuracy"), keras.metrics.AUC(name="auc"),
                           keras.metrics.AUC(curve="PR", name="pr_auc")])
    return model


def callbacks():
    return [keras.callbacks.EarlyStopping(monitor="val_auc", mode="max", patience=15, restore_best_weights=True),
            keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=6, min_lr=1e-5)]


grid = [dict(hidden=h, dropout=d, lr=lr) for h in [(32,), (64, 32), (128, 64, 32)]
        for d in [0.2, 0.4] for lr in [1e-3, 3e-4]]
if FAST:
    grid = grid[2:4]
EPOCHS = 30 if FAST else 150

tuning, best = [], None
t0 = time.time()
for i, cfg in enumerate(grid):
    keras.utils.set_random_seed(SEED)
    model = build_mlp(X_tr.shape[1], **cfg)
    hist = model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=EPOCHS, batch_size=128,
                     class_weight=class_weight, callbacks=callbacks(), verbose=0)
    pv_ = model.predict(X_va, verbose=0).ravel()
    row = {"Config": i + 1, "Hidden layers": "-".join(map(str, cfg["hidden"])), "Dropout": cfg["dropout"],
           "Learning rate": cfg["lr"], "Epochs run": len(hist.history["loss"]),
           "Best epoch": int(np.argmax(hist.history["val_auc"]) + 1),
           "Val ROC-AUC": r(roc_auc_score(y_va, pv_), 4), "Val PR-AUC": r(average_precision_score(y_va, pv_), 4),
           "Params": int(model.count_params())}
    tuning.append(row)
    print(row)
    if best is None or row["Val ROC-AUC"] > best["row"]["Val ROC-AUC"]:
        best = {"row": row, "cfg": cfg, "model": model, "history": hist.history}
tuning = pd.DataFrame(tuning)
tuning.to_csv(RES_DIR / "hyperparameter_tuning.csv", index=False)
RESULTS["tuning"] = tuning.to_dict(orient="records")
RESULTS["tuning_seconds"] = r(time.time() - t0, 1)
RESULTS["best_config"] = best["row"]
model = best["model"]
model.save(RES_DIR / "best_mlp.keras")
model.summary()
```

B.9 Grouped permutation importance

```
p_va = model.predict(X_va, verbose=0).ravel()
base_auc = roc_auc_score(y_va, p_va)
rng = np.random.default_rng(SEED)
imp_rows = []
for name, cols_ in groups.items():
    drops = []
    for _ in range(10):
        Xp = X_va.copy()
        perm = rng.permutation(len(Xp))
```

```
        Xp[:, cols_] = X_va[perm][:, cols_]          # shuffle the whole group jointly
        drops.append(base_auc - roc_auc_score(y_va, model.predict(Xp, verbose=0).ravel()))
    imp_rows.append({"Feature": name, "Mean AUC drop": float(np.mean(drops)), "SD": float(np.std(drops))})
imp = pd.DataFrame(imp_rows).sort_values("Mean AUC drop", ascending=False).reset_index(drop=True)
imp.to_csv(RES_DIR / "permutation_importance.csv", index=False)
```

B.10 Threshold selection and evaluation metrics

```
hist = best["history"]
p_te = model.predict(X_te, verbose=0).ravel()
prec_v, rec_v, thr_v = precision_recall_curve(y_va, p_va)
f1_v = 2 * prec_v[:-1] * rec_v[:-1] / np.maximum(prec_v[:-1] + rec_v[:-1], 1e-12)
thr_best = float(thr_v[np.argmax(f1_v)])          # threshold chosen on validation data only

logreg = LogisticRegression(class_weight="balanced", max_iter=5000, random_state=SEED).fit(X_tr, y_tr)
p_te_lr = logreg.predict_proba(X_te)[:, 1]
p_va_lr = logreg.predict_proba(X_va)[:, 1]
pl_, rl_, tl_ = precision_recall_curve(y_va, p_va_lr)
f1_l = 2 * pl_[:-1] * rl_[:-1] / np.maximum(pl_[:-1] + rl_[:-1], 1e-12)
thr_lr = float(tl_[np.argmax(f1_l)])


def metrics(y, p, t):
    yhat = (p >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y, yhat).ravel()
    return {"threshold": r(t, 3), "accuracy": r(accuracy_score(y, yhat), 4),
            "balanced_accuracy": r(balanced_accuracy_score(y, yhat), 4),
            "precision": r(precision_score(y, yhat), 4), "recall": r(recall_score(y, yhat), 4),
            "specificity": r(tn / (tn + fp), 4), "f1": r(f1_score(y, yhat), 4),
            "mcc": r(matthews_corrcoef(y, yhat), 4), "roc_auc": r(roc_auc_score(y, p), 4),
            "pr_auc": r(average_precision_score(y, p), 4), "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp":
int(tp)}
```